



INVESTIGACIÓN X

PROFUNDIZACIÓN ____

METAMODELO SOFTWARE PARA SIMULACIÓN REMOTA INTERACTIVA EN AMBIENTE WEB

Autor

Jeisson Rodrigo Piñeros Ramirez. Ing.

Director

Julio Barón Velandia. PhD.

Codirector

Sebastian Camilo Vanegas Ayala. PhD.

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS
Maestría en Ciencias de la Información y las Comunicaciones
Énfasis en Ingeniería de Software
Bogotá, Colombia
mayo de 2025

Contenido

Resumen	6
Palabras clave	6
Introducción	7
1 Problema de Investigación.....	9
1.1 Planteamiento del Problema.....	9
1.2 Formulación del Problema	12
1.3 Sistematización del problema	12
2 Objetivos	13
2.1 Objetivo General.....	13
2.2 Objetivos Específicos	13
3 Justificación	14
3.1 Justificación Teórica	14
3.2 Justificación Metodológica	14
3.3 Justificación Práctica	15
4 Marco de Referencia	16
4.1 Marco Conceptual.....	16
4.1.1 Simulación	16
4.1.2 Tipos de simulación	17
4.1.3 Interactividad	18
4.1.4 Procesamiento remoto.....	20
4.1.5 Web.....	21

4.1.6	Modelo de software	22
4.1.7	Metamodelo de software	22
4.1.8	Arquitectura Dirigida por Modelos.....	23
4.1.9	Ingeniería Dirigida por Modelos	24
4.1.10	Meta-Object Facility	25
4.1.11	Patrones de diseño.....	26
4.1.12	Eclipse Modeling Framework.....	27
4.1.13	Transformación de metamodelos.....	28
4.1.14	Lenguaje Unificado de Modelado.....	29
4.1.15	Proceso iterativo e incremental.....	29
4.2	Marco Teórico.....	30
5	Hipótesis.....	34
6	Metodología.....	35
6.1	Obtención de requerimientos.....	35
6.2	Modelado estructural	35
6.3	Modelado comportamental	35
6.4	Verificación.....	36
6.5	Actividades iterativas	36
7	Alcance y limitaciones	37
7.1	Alcance.....	37
7.2	Limitaciones.....	37
8	Metamodelo.....	38
8.1	Requisitos.....	39
8.2	Eventos de la simulación	43
8.3	Operaciones de la simulación	46

8.3.1	Comandos generales de simulación	48
8.3.2	Comandos para elementos simulados	49
8.3.3	Comandos de visualización	50
8.3.4	Envío y recepción de comandos	52
8.3.5	Metamodelo de operaciones de simulación	53
9	Transformación y validación	54
9.1	Transformación.....	54
9.1.1	Transformaciones para el servidor.....	55
9.1.2	Transformación para el cliente.....	57
9.1.3	Código específico	58
9.2	Validación	58
9.2.1	Modelo específico para simulación de redes	58
9.2.2	Pruebas y validación de prototipo	60
9.2.3	Resultados	61
10	Conclusiones	62
11	Aportes y trabajo futuro	63
12	Referencias.....	64

RESUMEN

La simulación desempeña un papel crucial en el apoyo a la comprensión de fenómenos del mundo real, la simulación interactiva con visualización progresiva de eventos es aún más valiosa para este fin, sin embargo, para ciertos tipos de sistemas se requiere de gran disponibilidad de recurso computacional, tanto en memoria como en procesamiento. Una posible solución a este problema es la realización de simulación en ambiente Web, dejando que el cliente realice la visualización progresiva de los resultados obtenidos según el procesamiento de la lógica que se realiza a nivel de servidor.

Las soluciones actuales de simulación presentan dificultades para los dispositivos cliente en relación con: bajo nivel de interactividad del usuario en el dispositivo cliente y alto consumo tanto de ancho de banda como de recursos computacionales para procesamiento de la lógica de simulación; haciendo que este tipo de soluciones limiten las posibilidades de interactividad o sean costosas con respecto al procesamiento, memoria y red. ¿Cómo disminuir los requisitos computacionales y de red en el cliente manteniendo altos niveles de interactividad y visualización progresiva en software de simulación?

El objetivo de esta propuesta es diseñar un metamodelo software para simulación remota interactiva con visualización progresiva en ambiente Web aplicando una técnica metodológica basada en modelos iterativos e incrementales que permiten obtener resultados en términos conceptuales y de desarrollo.

PALABRAS CLAVE

Metamodelo, Simulación, Procesamiento remoto, Interactividad, Ambiente Web, MDA

INTRODUCCIÓN

La simulación interactiva se ha consolidado como una opción fundamental para entender fenómenos que son de fácil comprensión a través de la visualización progresiva, permite experimentar con sistemas que, de otro modo, serían costosos o inseguros de replicar en el mundo real. Sin embargo, cuando el número de elementos y relaciones crece, los requisitos de procesamiento, memoria y ancho de banda aumentan de forma vertiginosa, degradando la experiencia de los usuarios con dispositivos de recursos computacionales limitados.

El entorno Web ofrece una solución mediante la centralización de carga computacional en el servidor, reduciendo la carga de lado del cliente mientras mantiene alto grado de interactividad. Sin embargo, las soluciones vigentes solo se enfocan en mejorar dos de las tres variables críticas para este tipo de simulaciones: interactividad, uso de red o recursos computacionales del cliente.

Este proyecto de investigación presenta los resultados obtenidos de un metamodelo software para simulación remota interactiva con visualización progresiva, actúa como una abstracción, formalizando los componentes fundamentales del cliente, del servidor y del canal de comunicación, integra patrones de diseño como observador, comando y fábrica abstracta con el fin de desacoplar responsabilidades y facilitar la reutilización. El modelo permite mantener un bajo nivel de utilización de recurso computacional en el dispositivo que visualiza la simulación y, simultáneamente, ofrece una base común sobre la cual implementar escenarios de simulación con alta demanda de interacción, procesamiento y ancho de banda.

El documento inicia con la descripción del problema de investigación, presenta los objetivos y la justificación que motivan la propuesta; luego se establecen los marcos conceptual y teórico que sustentan el proyecto, se plantea la hipótesis y la descripción metodológica basada en el modelo de proceso iterativo e incremental. A continuación, se detalla el alcance y las limitaciones, para dar paso la definición del metamodelo, sus requisitos y la

forma en que integra eventos y operaciones de simulación, posteriormente se detallan las transformaciones para llevar el modelo a código, seguidas del proceso de validación experimental y sus resultados; en el cierre se presentan las conclusiones, aportes, líneas de trabajo futuro, y finalmente las referencias bibliográficas que soportan la investigación.

1 PROBLEMA DE INVESTIGACIÓN

1.1 PLANTEAMIENTO DEL PROBLEMA

El uso de la simulación computacional permite replicar y analizar fenómenos complejos de situaciones reales o hipotéticas, proporcionando una forma de experimentar en entornos controlados, en particular, las simulaciones interactivas, como las discutidas por Knoblauch, se destacan por su capacidad para involucrar activamente a los usuarios, permitiéndoles manipular parámetros y observar los resultados de forma casi inmediata, esto fomenta el aprendizaje a través de la experiencia y la facilidad para la comprensión del fenómeno simulado (Knoblauch, 2024), la simulación interactiva en ambiente Web potencia la accesibilidad y facilidad para la comprensión de fenómenos en campos como la educación, la ingeniería y la investigación científica, brindando un entorno amigable y facilitando la adopción de conocimiento de las situaciones simuladas de manera práctica (Rorich, Bernhard, Handte, et al., 2014).

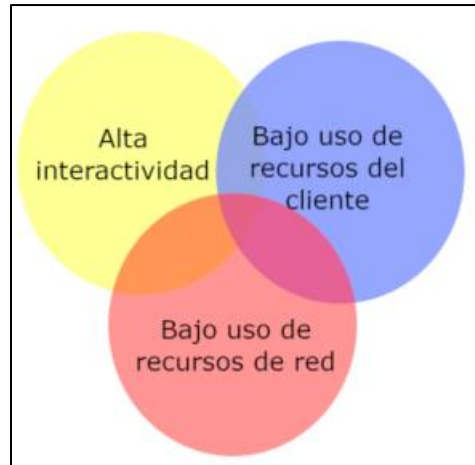
La realización de simulaciones interactivas con un gran número de elementos simulados, relaciones y variables, presenta desafíos relacionados a los altos requerimientos computacionales, que restringe la accesibilidad en dispositivos con recursos computacionales limitados, esta restricción de accesibilidad, se magnifica cuando se busca mantener una experiencia altamente interactiva, lo cual requiere estrategias de comunicación que además de considerar la capacidad de procesamiento del cliente implican el uso de red y el correcto funcionamiento general del sistema de simulación.

Las técnicas actuales para simulación remota han intentado abordar estos desafíos, pero con limitaciones, por ejemplo, algunas soluciones transmiten imágenes procesadas desde el servidor al cliente para mantener la interactividad y reducir la carga en este último, no obstante, esto implica un uso intensivo del ancho de banda y un procesamiento considerable en ambos extremos (Velichko, 2018), por otro lado, otras técnicas priorizan la centralización del procesamiento en el servidor y envían al cliente resultados preprocesados, lo cual minimiza el uso de recursos del cliente pero reduce la interactividad

en tiempo real (Koch et al., 2018), la falta de equilibrio entre interactividad, uso de recursos del cliente y ancho de banda, resalta un vacío en las soluciones existentes, que suelen optimizar solo dos de estas tres variables (Ng et al., 2008).

A medida que la demanda de recursos de procesamiento y el uso de red para aplicaciones interactivas aumenta (Mondesire et al., 2018), se vuelve esencial establecer nuevas propuestas que permitan optimizar estos factores sin comprometer la calidad de la experiencia del usuario. En este contexto, se evidencia la necesidad de una técnica de abstracción que permita estructurar la simulación de forma modular y adaptable. Los metamodelos han surgido como herramientas clave para representar sistemas complejos de manera flexible y eficiente, permitiendo la creación de entornos de simulación que se ajustan a diferentes contextos y necesidades (St-Aubin, 2022).

Teniendo en cuenta los casos anteriores que utilizan técnicas convencionales de comunicación para la interacción con el entorno de simulación es posible identificar las relaciones existentes entre la interactividad, el procesamiento de la simulación y los recursos de red desde el punto de vista del cliente. En la Ilustración 1 se observa el diagrama que relaciona estas variables, donde la intersección de las tres representa la situación ideal, sin embargo, los actuales desarrollos solo permiten hacer énfasis en dos de los tres aspectos.

Ilustración 1 Relación entre variables

Fuente: El autor

La demanda de recursos de procesamiento y recursos de red para la interactividad multimedia de los usuarios con los sistemas ha venido creciendo a través del tiempo (Mondesire et al., 2018), por lo que si se siguen utilizando las mismas técnicas sin modelado de simulación de extremo a extremo, el software interactivo consumirá cada vez más recursos computacionales y recursos de red, saturando canales de comunicaciones e imposibilitando la simulación de ciertos fenómenos en dispositivos con baja disponibilidad de recurso computacional.

En situaciones donde se aplica la simulación de forma intensiva son frecuentes los requisitos computacionales, por consiguiente, establecer una representación que pueda ser aplicada en diferentes contextos de simulación en ambiente Web, implica el establecimiento de un metamodelo que especifique funcionalidades de: el cliente, el canal y el servidor que reduzca la probabilidad de saturación del ancho de banda del canal, permita que un mayor número de dispositivos cliente puedan acceder a las funcionalidades del servidor de manera concurrente, sin que se requieran altos niveles de procesamiento computacional en los dispositivos finales.

1.2 FORMULACIÓN DEL PROBLEMA

¿Cómo representar en entorno computacional, diferentes modelos de simulación que exigen altos niveles de procesamiento, interactividad y visualización progresiva en ambiente Web, en escenarios que presentan restricciones en costos y disponibilidad de ancho de banda?

1.3 SISTEMATIZACIÓN DEL PROBLEMA

Dar solución al problema de investigación implica encontrar respuesta a las siguientes preguntas:

- ¿Cuáles son los requerimientos que debe cumplir un entorno de software de simulación remota en términos de funcionalidad, velocidad, interactividad, uso de red y recursos de procesamiento computacional?
- ¿Cuál debe ser la estructura del software de simulación remota que cumple con las necesidades interactivas utilizando el mínimo de recursos computacionales y de red?
- ¿Cómo se debe comportar el software de simulación remota para que permita alta interactividad cumpliendo con los requerimientos de uso de red y recursos computacionales?
- ¿Qué escenarios de simulación Web permiten verificar mediante métricas el cumplimiento de requerimientos de interactividad, visualización progresiva y procesamiento remoto?

2 OBJETIVOS

2.1 OBJETIVO GENERAL

Diseñar una representación general de simulación Web, aplicando metamodelos de software que permita especializar diferentes escenarios con altas exigencias de procesamiento, interactividad y visualización progresiva cuando se presentan restricciones en costos y disponibilidad de ancho de banda.

2.2 OBJETIVOS ESPECÍFICOS

- Establecer los requerimientos que debe cumplir el metamodelo software mediante la revisión de herramientas de simulación existentes, así como el análisis de técnicas de comunicación, procesamiento e interactividad para establecer las características mínimas que deben ser incluidas.
- Determinar a nivel estructural la representación que responde a los requerimientos a través de la aplicación de técnicas de comunicación y procesamiento, de acuerdo con las necesidades de interactividad con bajo uso de recursos computacionales y de red para el cliente.
- Consolidar el comportamiento de los escenarios de simulación a través de la síntesis de interactividad y procesamiento remoto, requeridos para facilitar la representación de diferentes tipos de interacción.
- Verificar la efectividad de simulación en ambiente Web mediante métricas de velocidad, usabilidad, uso de recursos computacionales y de red, en la representación de escenarios para confirmar alta interactividad, visualización progresiva y procesamiento remoto, manteniendo la experiencia de usuario en distintos tipos de dispositivos cliente.

3 JUSTIFICACIÓN

A continuación, se presenta la justificación en términos de los beneficios que conlleva el establecimiento de un marco conceptual abstracto para el manejo de sistemas interactivos, la utilización de una metodología personalizada para llegar a metamodelos y las ventajas de disminuir la utilización de recursos de red y procesamiento en el cliente manteniendo el nivel de interactividad en software de simulación.

3.1 JUSTIFICACIÓN TEÓRICA

El establecimiento de un marco conceptual enfocado al manejo de simulaciones interactivas en ambiente de red proporciona una perspectiva teórica sobre la que fundamentar el modelamiento de software interactivo en Web.

La caracterización de elementos abstractos en el diseño del mecanismo de captura de eventos de interacción y su modelamiento, proporcionan la base para la representación de escenarios en software con altas exigencias de procesamiento e interactividad.

El modelamiento abstracto del comportamiento dinámico del sistema como parte fundamental de la interacción entre usuario y cliente, profundiza en la teoría de la interacción y el comportamiento a nivel del meta-modelado y la generación de modelos específicos a partir de este tipo de abstracciones.

3.2 JUSTIFICACIÓN METODOLÓGICA

La técnica metodológica utilizada para el levantamiento de requerimientos a través de la síntesis de problemas con modelado y requerimientos propios permite contar con un método aplicable a otros contextos donde se requiere levantar requerimientos a través de la síntesis de problemas con requerimientos y modelado similar.

Las actividades y dinámicas que se apliquen para llegar a abstracciones de modelos de software establecen los mecanismos y la forma de aplicarlos para abordar problemas en los que se desea modelar de manera abstracta sistemas sobre diferentes contextos donde es pertinente la utilización de metamodelos.

3.3 JUSTIFICACIÓN PRÁCTICA

Al obtener representaciones que disminuyan las necesidades de los clientes a nivel de procesamiento computacional y de recursos de red, mediante alta disponibilidad de características de interacción, es posible proporcionar implementaciones software de simulación interactiva en ambiente Web que permite a un mayor número de usuarios acceder a servicios de simulación interactiva de manera concurrente.

Una representación abstracta de modelos de simulación remota permite generar modelos concretos que son útiles en escenarios de educación, física, química, procesos industriales, sistemas de transporte, climatológicos, entre otros.

La caracterización abstracta de la relación entre los nodos permite distribuir la carga computacional acorde a las características y servicios que requieren, por lo tanto, los recursos computacionales pueden ser explotados en mayor medida por usuarios en diferentes ubicaciones geográficas.

4 MARCO DE REFERENCIA

Este capítulo contiene el marco conceptual que trata un conjunto de conceptos de importancia para el proyecto y su relación con lo aplicado durante el desarrollo de este documento; también se plantea el marco teórico que contiene el estado actual de desarrollo, estudios y avances más relevantes sobre el problema de investigación aquí planteado.

4.1 MARCO CONCEPTUAL

El marco conceptual proporciona la base teórica que sustenta la investigación, orienta el desarrollo del proyecto y facilita la comprensión del problema de investigación al clarificar los conceptos clave y establecer las relaciones entre ellos (Hernández Sampieri et al., 2014), esto permite delimitar el campo de estudio y fundamentar metodológicamente el proyecto.

Se abordan los fundamentos teóricos y tecnológicos necesarios para el diseño de un metamodelo de software, que permita contar con modelos de simulación remota interactiva en ambiente Web, en los siguientes apartados, se exploran conceptos como la simulación, el modelado, el metamodelado, la arquitectura dirigida por modelos, la interactividad, entre otros, para establecer una base conceptual sólida en el desarrollo del proyecto.

4.1.1 Simulación

La simulación es una herramienta aplicada en múltiples disciplinas, utilizada para imitar o replicar el comportamiento de sistemas reales o hipotéticos a través de modelos computacionales. El término "simulación" ha sido empleado desde hace décadas, especialmente desde el desarrollo de los primeros computadores electrónicos durante la Segunda Guerra Mundial, cuando se utilizó para resolver problemas complejos en física nuclear mediante el Método de Monte Carlo (Metropolis & Ulam, 1949).

Según Banks et al., la simulación es "el proceso de diseñar un modelo de un sistema real y llevar a cabo experimentos con este modelo con el propósito de entender el comportamiento

del sistema o evaluar diversas estrategias para la operación del sistema" (Banks et al., 2014), esta definición destaca la simulación como una representación abstracta que permite experimentar y analizar sistemas sin intervenir directamente en ellos.

4.1.2 Tipos de simulación

Existen diversos tipos de simulación, cada uno adaptado a las características y necesidades del sistema a estudiar. A continuación, se describen los principales tipos:

4.1.2.1 Simulación Discreta

Se enfoca en sistemas donde los cambios de estado ocurren en puntos específicos en el tiempo. Es especialmente útil en el modelado de procesos donde los eventos son discretos y separados, como líneas de producción, sistemas de colas y redes de comunicación (Law, 2015).

4.1.2.2 Simulación Continua

Aplica para sistemas donde los cambios son continuos a lo largo del tiempo. Se utilizan modelos matemáticos basados en ecuaciones diferenciales para representar fenómenos físicos como el flujo de fluidos, la dinámica de sistemas y procesos químicos (Cellier & Kofman, 2006) .

4.1.2.3 Simulación Híbrida

Combina elementos de la simulación discreta y continua, permitiendo modelar sistemas que presentan ambos tipos de comportamiento. Esto es útil en áreas como la ingeniería de control y la mecatrónica, donde los sistemas físicos interactúan con sistemas digitales (Zeigler et al., 2000).

4.1.2.4 Simulación Interactiva

Es aquella en la que el usuario puede influir en el comportamiento del modelo en tiempo real, modificando parámetros y observando los resultados de manera inmediata. La simulación interactiva permite una mayor comprensión y exploración de sistemas complejos al involucrar activamente al usuario en el proceso de simulación (Cassenti, 2018).

4.1.2.5 Simulación Remota

Implica el acceso y control de simulaciones que se ejecutan en un servidor distante, permitiendo a los usuarios interactuar con el modelo desde diferentes ubicaciones geográficas a través de redes de comunicación, esto es particularmente útil cuando los recursos computacionales necesarios son significativos o cuando se requiere colaboración entre múltiples usuarios (Fujimoto, 2015).

4.1.2.6 Simulación basada en la Web

Permite a los usuarios acceder y ejecutar simulaciones a través de navegadores de Internet, utilizando tecnologías Web sin necesidad de instalar software especializado en sus dispositivos, esta aproximación aprovecha la ubicuidad y accesibilidad de la Web para distribuir aplicaciones de simulación de manera eficiente y escalable. Es esencial para democratizar el acceso a herramientas de simulación avanzadas, especialmente en entornos educativos y de investigación, donde los recursos pueden ser limitados y es clave para integrar simulaciones con otras tecnologías Web, como servicios en la nube y aplicaciones móviles, ampliando su alcance y funcionalidad (Taylor et al., 2015).

La simulación interactiva con procesamiento y acceso remoto es la que presenta mayor pertinencia para este proyecto, ya que es aplicable a campos como la educación, el entrenamiento y los videojuegos, donde la interactividad mejora la experiencia de aprendizaje y la toma de decisiones, requiriendo su implementación en un entorno Web.

4.1.3 Interactividad

La interactividad se refiere al grado en que los usuarios pueden influir en la forma o el contenido del entorno mediado en tiempo real. En el contexto de software y aplicaciones Web, la interactividad permite a los usuarios involucrarse activamente con el sistema, conduciendo a una experiencia más dinámica y receptiva (Steuer, 1992).

La interactividad es un componente clave de la experiencia de usuario (UX), ya que afecta la percepción de control y compromiso del usuario con el sistema. Altos niveles de interactividad pueden mejorar la satisfacción y eficacia del usuario (Nielsen, 1993).

Minguiell explica la comunicación de un usuario con un sistema a través de dos niveles de interactividad, el nivel de participación selectiva donde el usuario se limita a cambiar una vista, navegar entre un conjunto de elementos visualizando el contenido, y el nivel de participación transformativa donde el usuario puede modificar con su actuación el valor de variables, agregar o eliminar elementos entre otras acciones de transformación (Minguiell, 2013).

4.1.3.1 Principios del diseño interactivo

A continuación, se listan los principios a seguir para diseñar sistemas interactivos (Soler-Adillon, 2012):

- Visibilidad: Los elementos clave deben estar a la vista para guiar al usuario.
- Retroalimentación: Informar al usuario sobre el resultado de sus acciones de forma inmediata.
- Posibilidades: Ofrecer opciones claras sobre lo que es posible hacer en la interfaz.
- Limitaciones: Restringir acciones para evitar errores y mejorar la usabilidad.
- Mapajes: Relación intuitiva entre controles y sus efectos en el sistema.
- Navegación: Facilitar el desplazamiento y ubicación dentro del sistema.
- Consistencia: Mantener patrones de diseño similares en toda la interfaz.
- Gestión del error: Ayudar al usuario a prevenir, entender y corregir errores.

4.1.3.2 Visualización progresiva

La visualización progresiva se refiere a técnicas donde el contenido visual se carga o renderiza de forma incremental, mejorando el rendimiento percibido y la interactividad, especialmente en aplicaciones Web con grandes conjuntos de datos o gráficos complejos (Heer & Robertson, 2007).

4.1.4 Procesamiento remoto

Estos son los conceptos importantes a tener en cuenta para este proyecto sobre procesamiento remoto:

4.1.4.1 Arquitectura Llamada-Retorno

La arquitectura de llamada-retorno es un modelo tradicional donde un cliente realiza una solicitud a un servidor y espera una respuesta antes de continuar el procesamiento. Este modelo de comunicación sincrónica es simple, pero puede conducir a ineficiencias en sistemas distribuidos (Tanenbaum, 2007).

4.1.4.2 Modelo Cliente-Servidor

En el modelo cliente-servidor, los clientes solicitan servicios y los servidores los proporcionan. Este modelo es la base de gran parte de la computación distribuida y es fundamental en la arquitectura web (Tanenbaum, 2007).

4.1.4.3 Procesamiento Distribuido

Se caracteriza por múltiples computadoras que trabajan juntas en una red para realizar tareas, mejorando el rendimiento y la escalabilidad (Tanenbaum, 2007).

4.1.4.4 Computación en la Nube

Proporciona recursos informáticos bajo demanda a través de internet, ofreciendo escalabilidad y flexibilidad para procesamiento y almacenamiento (Mell & Grance, 2011).

4.1.4.5 Comunicación en tiempo real y protocolos

La comunicación en tiempo real se refiere a la comunicación en tan cortos periodos de tiempo que se vuelven imperceptibles para el usuario. (Loreto & Romano, 2014). A continuación, algunos protocolos y tecnologías que tienen una respuesta de este tipo.

- WebSockets: Un protocolo que proporciona canales de comunicación bidireccional sobre una única conexión TCP, permitiendo transferencia de datos en tiempo real a nivel de cliente y servidor.
- HTTP: El protocolo fundamental para la comunicación de datos en la Web, soportando servicios RESTful.

- REST (Representational State Transfer): Un estilo arquitectónico para diseñar aplicaciones en red utilizando comunicación sin estado y métodos HTTP estándar.
- SOAP (Simple Object Access Protocol): Un protocolo para intercambiar información estructurada en servicios web, utilizando XML.
- SSE (Server-Sent Events): Una tecnología donde un servidor envía actualizaciones automáticas al cliente vía HTTP.
- WebRTC (Web Real-Time Communication): Un proyecto de código abierto que permite comunicación en tiempo real (audio, video, datos) directamente entre navegadores web.

4.1.5 Web

La World Wide Web (WWW), comúnmente conocida como la Web, es un sistema de información donde documentos y otros recursos Web son identificados por Localizadores Uniformes de Recursos (URLs), interconectados por hipervínculos y accesibles a través de internet (Berners-Lee, Hendler, & Lassila, 2001).

4.1.5.1 Tecnologías Web

- HTML (HyperText Markup Language): El lenguaje de marcado estándar para crear páginas y aplicaciones web.
- CSS (Cascading Style Sheets): Un lenguaje de hojas de estilo utilizado para describir la presentación de un documento escrito en HTML o XML.
- JavaScript: Un lenguaje de programación interpretado de alto nivel que permite páginas web interactivas y es una parte esencial de las aplicaciones web.
- WebAssembly: Un formato binario de instrucciones para una máquina virtual basada en pila, diseñado para ejecutarse a velocidad casi nativa aprovechando las capacidades comunes del hardware.

4.1.5.2 *Diseño Responsivo*

Es un enfoque que sugiere que el diseño y desarrollo deben responder al comportamiento y entorno del usuario basado en el tamaño de pantalla, plataforma y orientación (Universidad Europea, 2024). Dentro de herramientas para el diseño responsivo se encuentran frameworks como Bootstrap o Materialize.

4.1.6 **Modelo de software**

Un modelo es una representación simplificada y abstracta de la realidad, que se utiliza para entender, analizar o predecir el comportamiento de un sistema o fenómeno. Un modelo posee tres características principales: es un mapeo de algo, es una reducción de la realidad y cumple una función pragmática para un propósito específico (Stachowiak, 1973).

En el contexto de la ingeniería de software, un modelo de software es una representación de ciertos aspectos de un sistema de software, con el objetivo de facilitar su comprensión, desarrollo y mantenimiento. Pueden representar estructuras, comportamientos, requisitos o cualquier otro aspecto relevante del software permitiendo visualizar, especificar, construir y documentar los artefactos de un sistema, sirviendo como una guía para el desarrollo efectivo y eficiente de software (Booch et al., 2000).

4.1.7 **Metamodelo de software**

Un metamodelo es un modelo que define el lenguaje y las reglas para la creación de otros modelos dentro de un dominio específico; proporciona las estructuras y semánticas necesarias para construir modelos conformes a él, estableciendo los conceptos, relaciones y restricciones que deben seguirse (Booch et al., 2000).

En ingeniería de software, los metamodelos son esenciales para normalizar la manera en que se representan y manejan los modelos, facilitando la interoperabilidad entre diferentes herramientas y plataformas. Por ejemplo, el Unified Modeling Language (UML) es un metamodelo ampliamente utilizado que define un conjunto especificaciones de elementos para modelar sistemas de software (Rumbaugh et al., 2000).

4.1.8 Arquitectura Dirigida por Modelos

Es una iniciativa estandarizada por el Object Management Group (OMG) que promueve el desarrollo de software basado en modelos de alto nivel de abstracción. El objetivo principal de la Arquitectura Dirigida por Modelos (MDA) es separar las especificaciones funcionales de un sistema de las implementaciones en plataformas tecnológicas específicas, permitiendo así la portabilidad, interoperabilidad y reutilización de los componentes del sistema (OMG, 2003).

La arquitectura dirigida por modelos propone tres niveles de abstracción o puntos de vista (“viewpoint”) que se describen a continuación:

4.1.8.1 Modelo Independiente de Computación (CIM - Computation Independent Model)

Describe el sistema desde la perspectiva del negocio, centrándose en los requisitos y el entorno operativo sin detalles computacionales. Permite a los stakeholders entender y validar los requisitos del sistema (OMG, 2003).

4.1.8.2 Modelo Independiente de Plataforma (PIM - Platform Independent Model)

Representa la arquitectura y funcionalidad del sistema sin considerar detalles de ninguna plataforma tecnológica específica. Proporciona una base sólida para la transformación hacia modelos específicos de plataforma (OMG, 2003).

4.1.8.3 Modelo Específico de Plataforma (PSM - Platform Specific Model)

Añade detalles técnicos y especificaciones de la plataforma en la que se implementará el sistema. Resulta de la transformación del PIM y está listo para ser utilizado en la generación de código o configuración de la plataforma (OMG, 2003).

4.1.8.4 Transformaciones y reglas de transformación

Las transformaciones en MDA son procesos que convierten modelos de un nivel de abstracción a otro, siguiendo reglas predefinidas. Estas transformaciones pueden ser las siguientes:

- Model-to-Model (M2M): Transformaciones entre modelos, como de PIM a PSM.
- Model-to-Text (M2T): Generación de código o documentación a partir de modelos.

Las reglas de transformación especifican cómo los elementos de un modelo fuente se convierten en elementos del modelo destino. (Warmer et al., 2003).

Estas reglas pueden ser definidas utilizando lenguajes de transformación como ATL (Atlas Transformation Language), QVT (Query/View/Transformation), Epsilon Transformation Language, entre otros.

4.1.9 Ingeniería Dirigida por Modelos

Utiliza modelos como artefactos primarios en todas las etapas del ciclo de vida del software. La Ingeniería Dirigida por Modelos (MDE) busca aumentar la productividad, calidad y mantenibilidad del software mediante la elevación del nivel de abstracción y la automatización de procesos a través de transformaciones de modelos (Schmidt, 2006). Es un paradigma más amplio que MDA, engloba diversas metodologías y herramientas orientadas al modelado. MDE facilita la creación de Lenguajes Específicos de Dominio (DSLs), la transformación y generación de modelos, y promueve la alineación entre los modelos y el código implementado.

Algunos conceptos relacionados con MDE y MDA que se usan con menor frecuencia se explican en la tabla Tabla 1.

Tabla 1 Conceptos relacionados a MDE

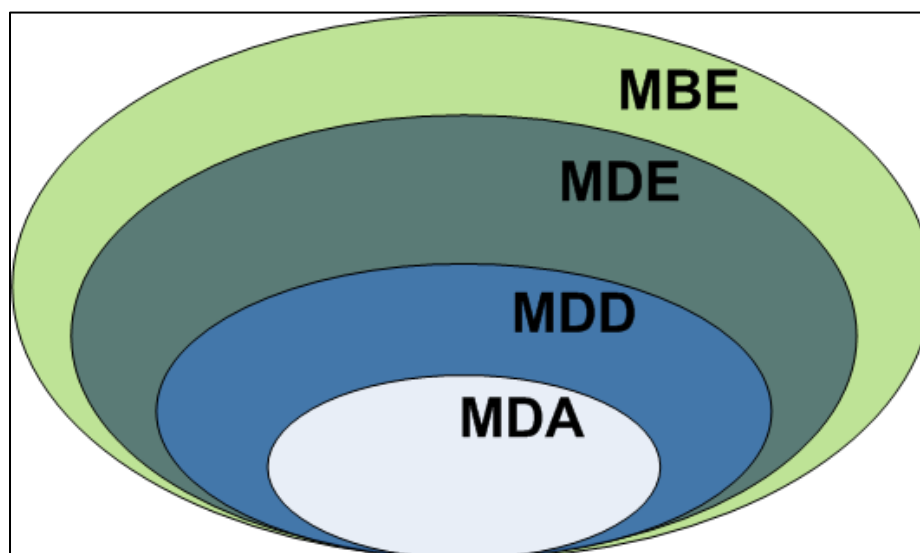
Concepto	Descripción	Propósito	Uso
MBE Model-based engineering	Uso de modelos en el ciclo de vida de sistemas complejos en cualquier disciplina de ingeniería	Visión general de sistemas complejos	General
MDE Model-driven engineering	Desarrollo de sistemas automatizado mediante transformaciones de modelos	Automatizar transformaciones y generación	Ingeniería de modelos
MDD Model-driven development	Desarrollo de software centrado en la generación de aplicaciones a partir de modelos	Productividad y reducción de errores en software	Desarrollo de software

Concepto	Descripción	Propósito	Uso
Model-driven architecture	Estándar de la OMG que define niveles de abstracción en modelos (CIM, PIM, PSM)	Portabilidad y separación de lógica y tecnología	Arquitectura de software

Fuente: Basado en (Brambilla et al., 2017)

La imagen Ilustración 2 resume la jerarquía de los conceptos mencionados en la tabla según Brambilla et al. (2017).

Ilustración 2 Jerarquía conceptos relacionado MDE



Fuente: (Brambilla et al., 2017)

4.1.10 Meta-Object Facility

Meta-Object Facility (MOF) Es un estándar del Object Management Group (OMG) que proporciona un marco para la definición de metamodelos y el manejo de metadatos, define un lenguaje y un conjunto de herramientas para crear, manipular y almacenar metamodelos, permitiendo la interoperabilidad entre diferentes sistemas y herramientas de modelado (OMG, 2019).

Este estándar cuenta con la posibilidad de tener diferentes capas o niveles de abstracción, en su última versión se argumenta que el número de capas es indefinido, sin embargo, lo habitual es contar con cuatro, que son las que se mencionan a continuación (OMG, 2019).

4.1.10.1 Nivel M3 (Meta-Metamodelo)

Es el nivel más abstracto y es usado por MOF para definirse a sí mismo, define las reglas y construcciones para crear metamodelos en el nivel M2.

4.1.10.2 Nivel M2 (Metamodelo)

Contiene metamodelos que definen lenguajes de modelado. Ejemplos incluyen el metamodelo de UML o Ecore en EMF.

4.1.10.3 Nivel M1 (Modelo)

Contiene modelos que son instancias de los metamodelos en M2. Por ejemplo, un diagrama de clases específico creado usando UML.

4.1.10.4 Nivel M0 (Datos o Instancias)

Representa los datos reales o instancias en tiempo de ejecución. Son objetos que son instancias de los modelos en la capa o nivel M1.

4.1.11 Patrones de diseño

Los patrones de diseño son soluciones reutilizables a problemas comunes en el diseño de software. Representan prácticas recomendadas que los desarrolladores pueden utilizar para resolver problemas de diseño recurrentes de manera consistente y eficiente (Gamma et al., 1996).

Los patrones de diseño se clasifican en tres grupos:

- Patrones Creacionales: Abordan mecanismos de creación de objetos, intentando crear objetos de manera adecuada a la situación.
- Patrones Estructurales: Se centran en la composición de objetos, formando estructuras más grandes a partir de componentes individuales.
- Patrones de Comportamiento: Se enfocan en la comunicación entre objetos, gestionando algoritmos y la asignación de responsabilidades.

Los patrones que se utilizan en este proyecto son:

4.1.11.1 Patrón comando

Patrón de comportamiento que encapsula una solicitud como un objeto, lo que permite parametrizar clientes con colas, solicitudes y operaciones. Desacopla el objeto que invoca la operación del que sabe cómo realizarla (Gamma et al., 1996).

4.1.11.2 Observador

Patrón de comportamiento que define una dependencia uno a muchos entre objetos, de manera que cuando un objeto cambia de estado, todos sus dependientes son notificados y actualizados automáticamente (Gamma et al., 1996).

4.1.11.3 Fábrica abstracta

Patrón creacional que proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas (Gamma et al., 1996).

4.1.12 Eclipse Modeling Framework

Es un marco de modelado y una herramienta de generación de código que facilita la construcción de aplicaciones basadas en modelos estructurados de datos. Eclipse Modeling Framework (EMF) permite a los desarrolladores generar código a partir de modelos proporcionando un entorno de soporte en tiempo de ejecución para dichos modelos. Integra el modelado con el desarrollo en Java, permitiendo la creación de aplicaciones con modelos de dominio enriquecidos (Steinberg et al., 2009).

EMF consta de varios elementos clave que se describen a continuación.

4.1.12.1 Ecore

Es el metamodelo central en EMF que define los constructos de modelado. Es esencialmente una implementación simplificada del estándar Meta-Object Facility (MOF) (Steinberg et al., 2009). Ecore define los siguientes elementos que pueden ser usados:

- Clases: Representan los tipos de objetos en el modelo.
- Atributos: Definen las propiedades de las clases.
- Relaciones: Establecen asociaciones entre clases.
- Operaciones: Definen comportamientos asociados con las clases.

4.1.12.2 GenModel

Es un modelo que almacena información del generador, especificando cómo se debe generar el código a partir del modelo Ecore (Steinberg et al., 2009).

4.1.12.3 EMF Edit

Proporciona clases reutilizables para construir editores para modelos EMF, incluyendo soporte para proveedores de contenido y etiquetas para visualizadores en árbol (Steinberg et al., 2009).

4.1.12.4 EMF Editor

Un editor generado que permite a los usuarios crear, editar y gestionar instancias del modelo EMF (Steinberg et al., 2009).

4.1.12.5 EMF Compare

Una herramienta para comparar y fusionar modelos EMF, útil para control de versiones y desarrollo colaborativo (Steinberg et al., 2009).

4.1.13 Transformación de metamodelos

Las herramientas y lenguajes de transformación son esenciales en el desarrollo dirigido por modelos para convertir modelos entre diferentes niveles de abstracción o formatos. A continuación, se mencionan algunas de las herramientas y lenguajes disponibles para la transformación de modelos.

4.1.13.1 ATL (Atlas Transformation Language)

Es un lenguaje de transformación de modelos y una herramienta desarrollada por la Fundación Eclipse. ATL se utiliza para escribir transformaciones de un modelo a otro, soportando transformaciones modelo a modelo (M2M) (Jouault et al., 2008).

4.1.13.2 Acceleo

Es un generador de código de código abierto que implementa el estándar de transformación modelo a texto (M2T) de la OMG. Acceleo permite a los desarrolladores generar código a partir de modelos EMF utilizando plantillas escritas en el lenguaje Acceleo (Vidal, 2016).

4.1.13.3 QVT (*Query/View/Transformation*)

Un conjunto de lenguajes definidos por la OMG para la transformación de modelos. Especifica tres lenguajes: Mappings Operacionales, Relaciones y Núcleo, que se pueden utilizar para especificar transformaciones entre modelos (OMG, 2019).

4.1.13.4 OCL (*Object Constraint Language*)

Un lenguaje formal utilizado para describir expresiones en modelos, particularmente para especificar invariantes, precondiciones y postcondiciones. OCL se utiliza en combinación con UML y otros lenguajes de modelado para proporcionar restricciones precisas (Warmer et al., 2003)

4.1.14 Lenguaje Unificado de Modelado

El Lenguaje Unificado de Modelado (UML) es un lenguaje de modelado estandarizado y de propósito general en ingeniería de software, que proporciona un conjunto de técnicas de notación gráfica para crear modelos visuales de sistemas de software orientados a objetos (OMG, 2015).

Los diagramas Clave de UML se agrupan en los siguientes dos tipos, cada uno contando con varias clases de diagramas.

- Diagramas Estructurales: Diagrama de clases, diagrama de componentes, diagrama de estructura compuesta, diagrama de despliegue, diagrama de objetos, diagrama de paquetes.
- Diagramas de Comportamiento: Diagrama de actividad, diagrama de secuencia, diagrama de máquina de estados, diagrama de casos de uso.

4.1.15 Proceso iterativo e incremental

Ya que mediante este proceso se aborda el proyecto, es necesario definir el modelo de proceso tanto iterativo como incremental.

Si hay una necesidad urgente de entregar una funcionalidad limitada a los usuarios y aumentarla en las entregas posteriores se elige un modelo de proceso diseñado para producir el software en incrementos. El modelo incremental aplica secuencias lineales en

forma escalonada a medida que avanza el calendario de actividades. Cada secuencia lineal produce “incrementos”, es frecuente que el primer incremento sea el producto fundamental. El modelo de proceso incremental se centra en que cada incremento se entrega un producto que ya opera (Pressman, 2010).

Los modelos iterativos se caracterizan por la manera en la que permiten desarrollar versiones cada vez más completas del software. El enfoque iterativo permite que el cliente evalúe en forma regular el aumento del software, dé la retroalimentación necesaria al equipo de software e influya en las adaptaciones del proceso que se realicen para aprovechar la retroalimentación (Pressman, 2010).

4.2 MARCO TEÓRICO

El desarrollo de simulaciones interactivas en entorno Web presenta desafíos en escenarios con limitaciones de requisitos computacionales y ancho de banda, la adopción de Model-Driven Architecture (MDA) y el uso de metamodelos han emergido como enfoques clave para mejorar la flexibilidad y adaptabilidad en la construcción de simulaciones, en este apartado se revisa de forma sistemática la literatura existente sobre metamodelos, simulaciones, interactividad y gestión de recursos en simulaciones Web, examinando tanto avances logrados como aspectos que requieren mayor investigación y desarrollo, evidenciando la pertinencia de este proyecto.

La aplicación de MDA y el uso de metamodelos han demostrado ser esenciales para brindar flexibilidad y adaptabilidad en la construcción de simulaciones, Deniz en "A Model Driven Approach to Web-based Traffic Simulation" demostró cómo el desarrollo dirigido por modelos permite la creación de simulaciones de tráfico en la Web, facilitando la modularidad y reutilización de componentes en entornos distribuidos, este enfoque reduce la dependencia de dispositivos específicos y permite la personalización de simulaciones según las características del cliente donde se interactúa con la simulación (Deniz, 2016).

Por otro lado en "Towards a Metamodel for Modular Simulation Environments" se explora la creación de metamodelos orientados a la interoperabilidad y modularidad en entornos de

simulación, se destaca la importancia de contar con componentes modulares que permiten a los desarrolladores integrar y adaptar simulaciones a necesidades específicas sin comprometer el rendimiento, estos principios son particularmente relevantes para simulaciones remotas que requieren adaptabilidad en escenarios con limitaciones de ancho de banda y procesamiento (Koch et al., 2018).

El diseño de simulaciones interactivas en un ambiente Web presenta desafíos específicos en términos de interactividad y recursos computacionales, se presenta la plataforma "OPTIMISE: An Internet-Based Platform for Metamodel-Assisted Simulation Optimization" que destaca la eficacia de los metamodelos en simulaciones remotas, utilizando una arquitectura modular que permite optimizar el procesamiento y la carga computacional en entornos distribuidos, demostrando cómo los metamodelos pueden mejorar el rendimiento en simulaciones Web (Ng et al., 2008).

El estudio "S-LPM: Segmentation Augmented Light-Weighting and Progressive Meshing for the Interactive Visualization of Large Man-Made Web3D Models" introduce técnicas de segmentación y visualización progresiva, que permiten la carga y visualización optimizadas en navegadores, un enfoque alineado con la necesidad de reducir la carga computacional en simulaciones interactivas en ambiente Web, siendo especialmente útiles en simulaciones con altos requerimientos visuales y mejorando la experiencia del usuario en términos de tiempo de respuesta en dispositivos con recursos limitados (Zhou et al., 2018).

En el artículo "Webdemos: An Interactive, Web-Based Visualization and Simulation Framework for Open Access", se evidencia que la interactividad es esencial en simulaciones para mejorar la experiencia de usuario y facilitar el control dinámico de parámetros, allí se plantea una plataforma que permite simulaciones en un entorno accesible, destacando la importancia de la interactividad para el aprendizaje y la visualización de datos en tiempo real (Rorich, Bernhard, Handte, et al., 2014), la estructura modular de esta herramienta, junto con su capacidad de facilitar la interacción con las simulaciones sin requerir infraestructura avanzada, respalda los objetivos de este proyecto, donde se busca maximizar la accesibilidad y el uso eficiente de recursos computacionales y de red en simulaciones Web.

En "Web VizOS: A Web-based Interactive and Visualized Approach to Simulations of Operating Systems" se examina una plataforma que permite la visualización y análisis de sistemas operativos en un entorno Web, su estructura modular facilita la personalización y adaptación de simulaciones complejas, promoviendo la interactividad y el uso razonable de recursos computacionales y de red en la Web (Pham, 2022), estos principios de modularidad y optimización pueden aplicarse directamente en el diseño de un metamodelo para simulación remota.

"Interactive Visualization of Agent-Based Pandemic Simulation in Web Browser" proporciona un ejemplo de cómo se pueden manejar grandes volúmenes de datos en simulaciones interactivas distribuidas en la Web, utilizando tecnologías como Three.js para la visualización inmediata y PostgreSQL para la gestión de datos geoespaciales (Mrozek et al., 2022), la escalabilidad y eficiencia de estos métodos pueden inspirar la mejora en la transmisión y visualización de datos en simulaciones remotas que impliquen múltiples variables y requisitos visuales.

En cuanto a los estándares de comunicación y distribución de datos, "Addressing the Challenge of Distributed Interactive Simulation With Data Distribution Service" analiza el uso de DDS (Data Distribution Service) para optimizar la comunicación en simulaciones interactivas distribuidas, el modelo de publicación/suscripción de DDS permite una transmisión eficiente de datos y garantiza la calidad de servicio (QoS), una característica fundamental para simulaciones remotas de respuesta inmediata (Hakiri et al., 2010); este enfoque es aplicable en la transmisión y sincronización de datos en simulación Web interactiva, mejorando el rendimiento en dispositivos con recursos computacionales o recursos de red limitados.

A pesar de los avances, aún existen vacíos significativos en la integración de entornos de simulación que combinen interactividad, modularidad, reutilización y disminución de recursos computacionales y de red en una plataforma unificada para simulaciones Web. "A Metamodel-Driven Interactive Framework for a Designer Assistance System" explora el diseño de un sistema interactivo basado en metamodelos, donde se facilita la

personalización en tiempo real de parámetros (Menzel et al., 2010), aunque se enfoca en el diseño industrial, los principios de modularidad, reutilización e interactividad son aplicables al metamodelo de simulación Web de este proyecto, donde la flexibilidad y personalización son necesarias para ajustarse a distintos escenarios de simulación.

La construcción de un metamodelo de simulación interactiva para ambiente Web se beneficia de la integración de técnicas de reutilización y modularidad descritas en los estudios anteriores, inspirado en las diferentes plataformas aquí mencionadas, este proyecto propone una estructura distribuida y reutilizable para proponer simulaciones en ambiente Web que facilitan la visualización progresiva y la interactividad en dispositivos cliente con limitaciones de recursos computacionales y de red, además, el uso de tecnologías Web como Angular y Node.js, mencionadas en artículos como el titulado "Web-Based Simulation Environment for Vehicular Electrical Networks" (Dominguez et al., 2021), respalda la viabilidad de implementar un sistema adaptable a diversos dispositivos y condiciones de ancho de banda.

En este apartado se ha identificado que el uso de metamodelos es beneficioso para el desarrollo de simulaciones Web interactivas que combinen eficiencia y adaptabilidad en contextos de limitaciones de ancho de banda y procesamiento. Los vacíos en la integración de interactividad, modularidad, reutilización y uso eficiente de recursos dentro de un metamodelo unificado de simulación resaltan la necesidad de un proyecto como el presente, que propone una solución innovadora para superar estos desafíos, el proyecto se orienta a desarrollar un sistema de simulación remota en ambiente Web que mantiene la interactividad y reduce el consumo de recursos computacionales, proporcionando una plataforma de simulación accesible y adaptable a múltiples escenarios y dispositivos.

5 HIPÓTESIS

Un metamodelo software para simulación remota interactiva que incorpore un mecanismo de captura de eventos y técnicas para el envío bidireccional de comandos reduce las necesidades computacionales a nivel de procesamiento, memoria y uso de red en el dispositivo cliente de visualización y facilita la creación de sistemas de simulación a través de la reutilización de una abstracción propuesta.

6 METODOLOGÍA

Para la realización del proyecto se hace uso de técnicas y métodos de ingeniería que comprende aspectos, deductivos, inductivos y empíricos, los cuales son aplicados mediante un proceso iterativo e incremental en cuatro fases, articulando las actividades a realizar mediante iteraciones hasta obtener un resultado satisfactorio.

6.1 OBTENCIÓN DE REQUERIMIENTOS

Se realiza análisis empírico de un conjunto de herramientas de simulación existentes a través de la observación y medición de características específicas, para que mediante el método inductivo se proponga el conjunto de requisitos con los que debe contar un software de simulación a nivel funcional en lo que respecta a estructura y comportamiento de la simulación y no funcional en lo que respecta a características de interactividad, velocidad, recursos computacionales necesarios y recursos de red o forma de comunicación según sea el caso.

6.2 MODELADO ESTRUCTURAL

Mediante el análisis de los requerimientos, dividiendo en partes el modelado que se debe realizar, se plantean un conjunto de módulos y componentes de modelado estático a los que es posible aplicar método de síntesis, generando así un modelo desde las partes que se completa al realizar la integración.

6.3 MODELADO COMPORTAMENTAL

Mediante el análisis de los requerimientos, dividiendo en partes el modelado que se debe realizar, se plantean un conjunto de módulos y componentes de modelado dinámico a los que es posible aplicar método de síntesis, generando así un modelo desde las partes complementarias una vez se realice la integración.

6.4 VERIFICACIÓN

Se realiza la implementación del metamodelo planteando escenarios que impliquen simulación interactiva con altas necesidades computacionales y mediante un análisis estadístico descriptivo e inferencial de variables como: velocidad de respuesta, recursos utilizados en el cliente, recursos utilizados en el servidor, ancho de banda y grado de interactividad (el cual es medido mediante una propuesta basada en bibliografía correspondiente) se compruebe si el metamodelo propuesto cumple con los requerimientos establecidos y responde a la pregunta de investigación.

6.5 ACTIVIDADES ITERATIVAS

Buscando un resultado mejor que el anterior a lo largo de cada una de las fases se itera sobre el siguiente conjunto de actividades:

- Obtención Bibliográfica
- Definición de requerimientos
- Análisis de requerimientos
- Diseño de software
- Implementación de software
- Pruebas unitarias y de integración

7 ALCANCE Y LIMITACIONES

7.1 ALCANCE

Como resultado del proyecto se contará con un metamodelo software en el cuál es posible generar modelos e implementaciones específicas de software de simulación discreta que atienda a altas necesidades de visualización, interactividad y procesamiento remoto con bajo consumo de recursos computacionales en el cliente.

7.2 LIMITACIONES

Teniendo en cuenta las restricciones de infraestructura computacional, el prototipo software producto de este proyecto será desplegado en una red de área local bajo la pila de protocolos TCP/IP v4, con una capacidad de procesamiento de 2GHz y 6GB de memoria RAM para fines de pruebas, por lo tanto, el número de usuarios que pueden generar modelo y ejecución en el ambiente de pruebas dependerá de los recursos disponibles.

El cliente requiere usar un navegador Web que soporte JavaScript, HTML 5 y protocolos de bajo nivel de latencia para comunicación con el servidor.

Para la verificación de los resultados de transformación de metamodelo a una implementación específica solo se aplicará sobre un escenario particular.

8 METAMODELO

En el ámbito de la Arquitectura Dirigida por Modelos (MDA), un metamodelo se concibe como una abstracción que define la estructura, semántica y restricciones de un conjunto de modelos, actuando como una "gramática" para la construcción de modelos específicos dentro de un dominio (Warmer et al., 2003). El Meta-Object Facility (MOF) es un estándar de la Object Management Group (OMG) que proporciona un marco para la definición de estos metamodelos de manera formal y consistente (OMG, 2019). En este proyecto, se desarrolla un metamodelo software orientado a la simulación remota interactiva en ambiente Web, alineado con los principios de MDA y utilizando MOF como base, lo que facilita la interoperabilidad y la reutilización de componentes en el desarrollo de aplicaciones de simulación.

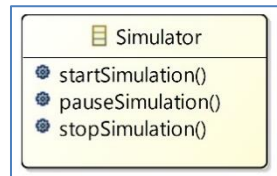
Para la creación y diseño del metamodelo, se hace uso de la plataforma Eclipse junto con el Eclipse Modeling Framework (EMF) y su lenguaje de modelado Ecore. EMF Ecore permite definir metamodelos y generar código automáticamente, agilizando el proceso de desarrollo y asegurando la conformidad con los estándares establecidos (Steinberg et al., 2009). Esta herramienta facilita el modelado eficiente de los componentes y relaciones necesarios para soportar la simulación remota interactiva en un entorno Web.

En Eclipse Modeling Framework (EMF), es fundamental definir una clase raíz en el metamodelo que actúe como contenedor de todas las demás clases mediante composición, permitiendo estructurar el modelo de forma coherente, facilitando la generación automática de código y asegurando que los elementos del modelo estén correctamente organizados y accesibles (Steinberg et al., 2009).

En este proyecto, la clase raíz definida es "Simulator", esta clase central agrupa todas las entidades necesarias para la simulación, incluyendo las clases relacionadas al manejo de eventos y la gestión de operaciones entre cliente y servidor que serán detalladas más adelante; en la Ilustración 3 se visualiza la clase con algunas operaciones generales para

el control de la simulación como lo son “startSimulation”, “pauseSimulation” y “stopSimulation”.

Ilustración 3 Clase raíz Simulator



Fuente: El autor

A continuación, se plantea la forma en la que se obtienen los requisitos que debe contemplar la abstracción propuesta, se detalla el diseño estructural y de comportamiento del metamodelo, mostrando la aplicación de patrones de diseño, como comando, observador y fábrica abstracta, estos patrones contribuyen a una solución modular y reutilizable para simulación remota interactiva en ambiente Web, integrando las clases necesarias en el metamodelo, y surtiendo así las clases que componen a la clase raíz “Simulator”.

8.1 REQUISITOS

El desarrollo de un metamodelo para simulación remota interactiva en un entorno Web debe contemplar un conjunto de requisitos de software que aseguren la funcionalidad, interactividad, manejo de eventos y la eficiencia en la comunicación entre cliente y servidor, estos requisitos permiten que el sistema ofrezca una experiencia de usuario robusta y optimizada, alineándose con las expectativas de los usuarios en términos de rendimiento y usabilidad.

En el contexto de la Arquitectura Dirigida por Modelos (MDA), la definición de estos requisitos corresponde al Modelo Independiente de la Computación (CIM), el CIM se centra en capturar las necesidades del negocio y los requisitos del sistema a un nivel abstracto sin involucrar detalles técnicos, esto permite una visión clara de las funcionalidades y condiciones que debe cumplir el sistema (Warmer et al., 2003).

A partir del análisis de soluciones existentes (identificadas en el marco teórico), las limitaciones identificadas en el planteamiento del problema y la literatura técnica revisada, se derivan los siguientes requisitos esenciales, que se describen a continuación agrupados por categorías para mayor claridad.

En cuanto a los requisitos funcionales y de simulación, la funcionalidad central del sistema debe permitir la ejecución y control efectivos de la simulación. Esto incluye operaciones básicas de gestión como iniciar, pausar y detener la simulación, las cuales son esenciales para cualquier entorno de este tipo, con el fin de manipular los elementos que componen la simulación, permitiendo su creación, eliminación y la modificación dinámica de sus propiedades y parámetros, es necesario contar con alta interactividad, característica clave destacada en referencias como presentan Knoblauch y Qui Wu (Knoblauch, 2024) (Qi Wu et al., 2024). El almacenamiento, recuperación, modificación y tratamiento de parámetros es igualmente importante para permitir la experimentación y el análisis de diferentes escenarios.

Con el fin de facilitar la comprensión del fenómeno simulado, es indispensable una visualización clara de los estados del sistema, implementada preferiblemente mediante técnicas de visualización progresiva, la cual implica cargar y mostrar los datos visuales de forma incremental para mejorar la percepción de velocidad y reducir la carga inicial en el cliente, así mismo, para asegurar la corrección y fiabilidad, el metamodelo y las herramientas asociadas deben proveer facilidades para la validación de los modelos generados.

La Interactividad es un pilar fundamental de este proyecto, buscando superar las limitaciones de las soluciones actuales, por ello, se requiere una interacción en tiempo real, donde el usuario pueda modificar parámetros y observar los efectos de manera inmediata o con mínima latencia perceptible lo cual se logra a través de una Interfaz Gráfica de Usuario (GUI) altamente interactiva, que permita manipular el entorno y los parámetros mediante controles intuitivos. El diseño de esta interfaz deberá adherirse a principios de diseño interactivo reconocidos, como la visibilidad, retroalimentación, manejo de

limitaciones y consistencia, identificados como importantes para garantizar una experiencia de usuario fluida.

Es fundamental que el procesamiento principal de la simulación resida en el servidor, siguiendo una arquitectura cliente-servidor recomendada en fuentes como (Ng et al., 2008) (Rorich, Bernhard, & Handte, 2014); esto permite centralizar los cálculos intensivos y asegurar compatibilidad con diversos dispositivos cliente, la comunicación entre cliente y servidor debe ser eficiente, minimizando la latencia y el volumen de datos transferidos. Una estrategia clave, identificada en los requisitos iniciales, es enviar actualizaciones solo cuando ocurren cambios significativos.

Se necesita establecer una comunicación bidireccional robusta que permita tanto las solicitudes del cliente como las notificaciones del servidor, necesaria para la interacción y sincronización. Todo esto contribuye al objetivo primordial de optimizar el uso de recursos computacionales, en términos de procesamiento, uso de memoria y de ancho de banda en el cliente.

Desde la perspectiva de diseño, la adopción de un enfoque basado en metamodelos busca inherentemente la modularidad, flexibilidad y reutilización, el metamodelo debe definir una estructura con componentes modulares que puedan ser reutilizados y extendidos fácilmente, como se propone en (Rorich, Bernhard, & Handte, 2014), facilitando la adaptación a diferentes dominios de simulación sin requerir modificaciones profundas.

Debe ser flexible para incorporar nuevos tipos de comandos, eventos o elementos de simulación, una necesidad identificada en los requisitos iniciales. Siguiendo los principios de MDA, se busca la independencia de plataforma en su definición como lo es el nivel PIM. El diseño debe basarse en el alto desacoplamiento, separando la lógica de simulación de la vista, los elementos observables de sus observadores, y los iniciadores de comandos de sus ejecutores, como se planteó en el análisis inicial de patrones, promoviendo así la mantenibilidad y la escalabilidad. Es importante que el metamodelo mantenga la independencia del contexto o dominio específico de simulación, siendo una abstracción general.

Además de los requisitos anteriores, existen requisitos no funcionales clave para el éxito del modelo, la accesibilidad Web es primordial, permitiendo el uso desde navegadores estándar sin barreras de instalación, lo que implica asegurar la compatibilidad multiplataforma con diversos sistemas operativos y dispositivos, soportando tecnologías Web modernas.

El rendimiento debe ser fluido, de modo que soporte la interacción continua del usuario sin interrupciones o retardos perceptibles. Aunque las pruebas iniciales se realizarán en un entorno limitado, la estructura debe diseñarse considerando la escalabilidad futura para manejar mayores cargas.

Estos requisitos, tanto en su forma desarrollada narrativamente como en la síntesis final, guían el diseño estructural y comportamental del metamodelo que se detalla en las secciones posteriores.

A continuación, se presenta una lista de los requisitos clave identificados para el metamodelo de simulación remota interactiva:

8.1.1.1 Funcionales y de Simulación

- Control general de la simulación (inicio, pausa, detención).
- Creación, eliminación y modificación de elementos y parámetros de simulación.
- Gestión de configuraciones de parámetros.
- Soporte para simulación basada en eventos discretos.
- Visualización clara de estados del sistema mediante técnicas progresivas.

8.1.1.2 Interactividad

- Interacción en tiempo real (baja latencia perceptible).
- GUI interactiva e intuitiva basada en principios de diseño interactivo.
- Retroalimentación constante e inmediata al usuario.
- Comunicación y procesamiento remoto.
- Procesamiento principal de la simulación en el servidor.
- Comunicación cliente-servidor eficiente (enviar solo cambios significativos).

- Comunicación bidireccional robusta.
- Optimización del uso de recursos en el cliente (CPU, memoria, red).
- Implementación de mecanismos de Calidad de Servicio (QoS).

8.1.1.3 *Diseño*

- Diseño modular, flexible y extensible.
- Reutilización de componentes.
- Independencia de plataforma (Nivel PIM - MDA).
- Alto desacoplamiento (lógica/vista, observable/observador, comando/receptor).
- Independencia del dominio específico de simulación.
- Aplicación de patrones de diseño (Comando, Observador, Fábrica Abstracta).
- Uso de formatos estandarizados (Ecore/XMI) para modelos/metadatos.

8.1.1.4 *No Funcionales*

- Accesibilidad vía navegador web estándar.
- Compatibilidad multiplataforma (incluyendo dispositivos móviles).
- Rendimiento fluido que soporte la interactividad.
- Diseño considerando la escalabilidad.
- Facilidades para la monitorización y obtención de métricas de verificación.

8.2 EVENTOS DE LA SIMULACIÓN

En el contexto de las simulaciones remotas interactivas, surge la necesidad de gestionar y capturar los eventos que ocurren dentro de la simulación, estos eventos pueden provenir de múltiples elementos que cambian de estado de manera independiente y asíncrona; una de las bases conceptuales que ha influido en el diseño de sistemas distribuidos y de procesamiento remoto es el modelo de publicación y suscripción utilizado por Data Distribution Service (DDS), este estándar permite una transmisión eficiente de datos con calidad de servicio, facilitando la sincronización y la distribución de eventos de manera que se minimiza el impacto en el rendimiento de los dispositivos cliente, especialmente en contextos con limitaciones de recursos computacionales y de red (Hakiri et al., 2010).

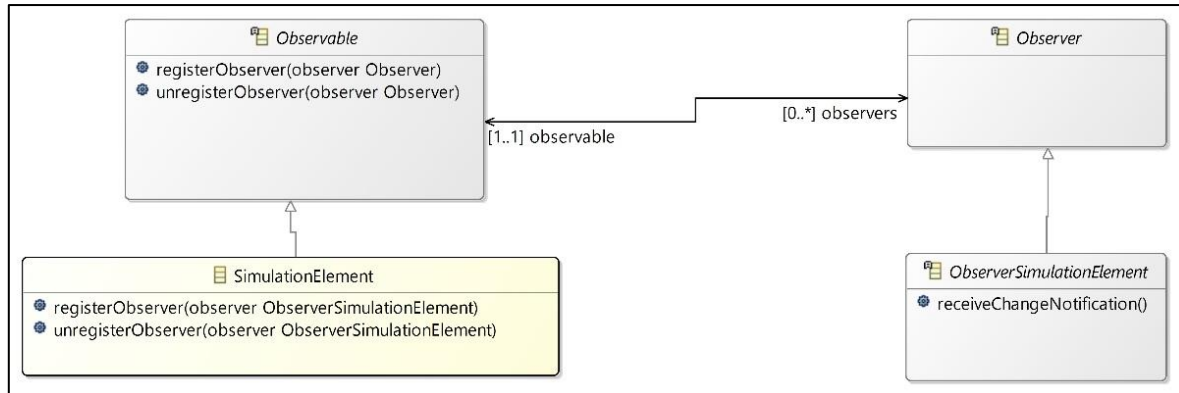
El patrón observador es un patrón de diseño que define una relación de dependencia uno a muchos entre objetos, de manera que cuando un objeto cambia de estado, todos sus dependientes son notificados y actualizados (Gamma et al., 1996), al igual que con DDS, el patrón observador permite notificar la ocurrencia de eventos de forma independiente y asíncrona a múltiples suscriptores, en este caso llamados observadores; ambos enfoques gestionan la difusión de cambios de estado, manteniendo un bajo acoplamiento entre el emisor y los receptores de dichos cambios.

Al igual que DDS, el patrón observador aplicado al metamodelo propuesto permite la creación de una infraestructura modular donde los observadores reaccionan a los eventos generados por los elementos de simulación sin necesidad de que estos elementos conozcan los detalles de sus observadores, mejorando la escalabilidad y la extensibilidad del sistema de simulación.

La estructura resultante de la aplicación del patrón observador en el metamodelo se detalla en la Ilustración 4, se creó una clase abstracta “Observable”, que representa cualquier objeto cuyo estado puede ser observado, a partir de esta clase, se derivó la clase concreta “SimulationElement”, la cual puede ser ampliada en sus atributos u operaciones, o de ser necesario es posible crear más clases concretas según lo requiera el contexto específico de cada modelo, esto permite que cada elemento de la simulación pueda ser observado y notifique a los observadores sobre cualquier cambio en su estado.

Se definió una clase abstracta “Observer”, de la cual surge la clase concreta “ObserverSimulationElement”, cada Observable está asociado mediante una relación de composición a un “Observer”, los observadores concretos como “ObserverSimulationElement” se encargan de ejecutar la operación responsable de recibir los cambios, en este caso a través de “receiveChangeNotification()” o la operación que corresponda.

Ilustración 4 Aplicación de patrón observador en el metamodelo

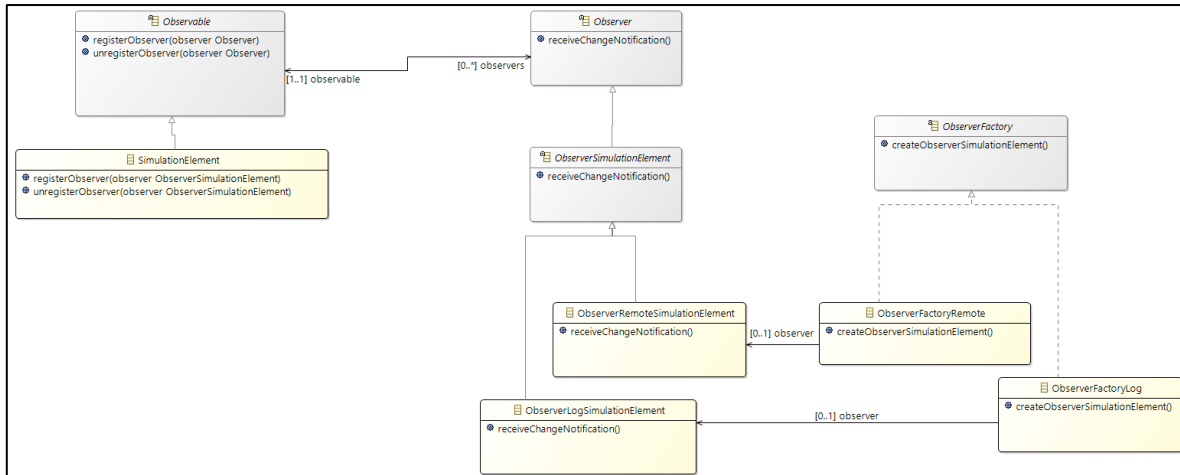


Fuente: El autor

En este contexto es posible requerir diferentes grupos o familias de observadores para cada uno de los elementos propios del contexto de simulación, cada grupo podría tener diferentes tipos de observadores, por ejemplo un observador concreto podría utilizarse para notificar al lado del cliente por un mecanismo específico de comunicación, otro observador para notificar a otro tipo de cliente por otro mecanismo de comunicación distinto, y alguno más para el guardado de histórico o log de eventos de simulación; para facilitar la creación y gestión de diferentes grupos de observadores, se implementa el patrón fábrica abstracta, que proporciona una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas (Gamma et al., 1996), en este metamodelo, se diseñan fábricas abstractas que permiten agrupar y disponer de distintos conjuntos de observadores que ejecutan acciones específicas.

En la Ilustración 5 se observa la aplicación del patrón de diseño fábrica abstracta en el metamodelo propuesto, ampliando la abstracción inicial para la captura de eventos de simulación, se crea la clase “**ObserverFactory**”, clase abstracta que tiene en este caso la operación “`createObserverSimulationElement()`” para crear un observador específico, para este caso se agregaron las clases concretas “**ObserverFactoryRemote**” y “**ObserverFactoryLog**” que se relacionan mediante referencia con “**ObserverRemoteSimulationElement**” y “**ObserverLogSimulationElement**”, permitiendo así que cada fábrica concreta pueda crear instancias de observadores concretos o familias de estos.

Ilustración 5 Aplicación de patrón fábrica abstracta en el metamodelo



Fuente: El autor

Nótese que el metamodelo es flexible y extensible de diferentes formas, teniendo la posibilidad de agregar múltiples elementos de simulación concretos que sean subtipos de “Observable”, elementos que representan los objetos dentro de la simulación, también es posible agregar múltiples observadores agrupados en múltiples familias por cada elemento existente en el modelo de simulación, agregando así observadores según lo que se requiera hacer cuando se notifique un evento de simulación.

La aplicación del patrón observador y fábrica abstracta en el metamodelo de simulación resuelve el problema de capturar y gestionar eventos de manera modular y escalable, los observadores pueden reaccionar a los cambios sin que los elementos observables necesiten conocer detalles de estos, promoviendo un bajo acoplamiento y alta cohesión en el diseño del sistema, al permitir la adición de nuevos tipos de observadores y fábricas, el metamodelo puede adaptarse a futuros requerimientos y diferentes contextos de simulación sin necesidad de reestructuraciones significativas.

8.3 OPERACIONES DE LA SIMULACIÓN

En simulaciones interactivas que operan en un entorno Web, con ejecución de operaciones de forma distribuida, es fundamental manejar una variedad de acciones que pueden ser

iniciadas tanto por el cliente como por el servidor, estas operaciones pueden abarcar desde el control general de la simulación hasta la manipulación detallada de elementos simulados individuales y la interacción con la interfaz gráfica, uno de los mayores retos es mantener una estructura que permita la flexibilidad necesaria para integrar nuevas operaciones sin modificar el núcleo de la aplicación, garantizando que las distintas acciones estén coordinadas entre cliente y servidor, logrando su correcta transmisión y ejecución siendo fácilmente extensibles.

El patrón comando es una solución de diseño que encapsula una acción como un objeto, separando el origen de la solicitud de su ejecución, esto permite que las operaciones se manejen de manera desacoplada y modular, promoviendo la extensibilidad y la reutilización de código (Gamma et al., 1996).

En el contexto de este proyecto, la implementación del patrón comando es de utilidad para gestionar de manera unificada y organizada las distintas operaciones de la simulación, tanto generales como específicas, además, facilita la comunicación y sincronización entre cliente y servidor, lo que es esencial para un entorno de simulación remota, garantizando que las acciones puedan transmitirse, codificarse, decodificarse y ejecutarse correctamente sin depender de métodos específicos.

Para la implementación del metamodelo, se define una clase abstracta denominada "Command", la cual contiene el atributo "parameters", encargado de almacenar los datos necesarios para ejecutar el comando, y la operación abstracta "execute()", que debe especificarse en cada comando concreto según su propósito, esta clase abstracta actúa como una interfaz común para los comandos del simulador, promoviendo la reutilización y estandarización de los comandos en todo el metamodelo.

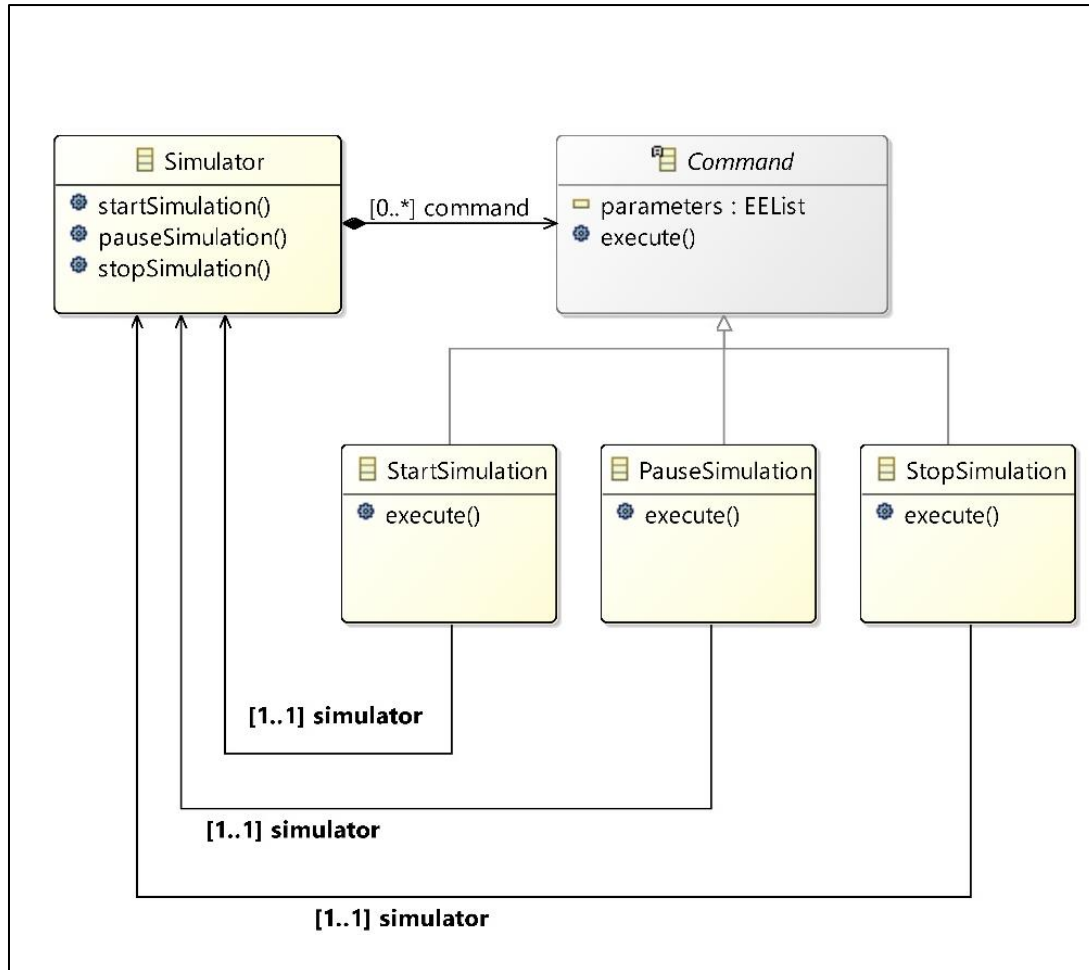
Para este proyecto se consideran tres tipos de comandos agrupados según su propósito, su receptor y el invocador del comando, a continuación, se detalla cada uno de estos grupos.

8.3.1 Comandos generales de simulación

El primer grupo de comandos concretos se ejecuta directamente sobre la clase Simulator, ya que son comandos generales que afectan el estado de la simulación en su totalidad. Estos comandos incluyen StartSimulation, PauseSimulation y StopSimulation. Dado que impactan el estado global, el receptor en estos casos es la misma clase Simulator, que es responsable de iniciar, pausar o detener la simulación según el comando recibido. Este grupo de comandos permite un control directo sobre el flujo de la simulación y resulta esencial para coordinar los estados de inicio y finalización, especialmente en un entorno remoto donde el control centralizado es necesario para sincronizar las instancias cliente y servidor.

En la Ilustración 6 se observa la clase Command, de la cuál heredan los comandos concretos, allí se encuentran algunos de los comandos generales de simulación como lo son StartSimulation, PauseSimulation y StopSimulation, los cuáles se relacionan directamente con la clase Simulator para hacer la ejecución que corresponda.

Ilustración 6 Comandos generales de simulación



Fuente: El autor

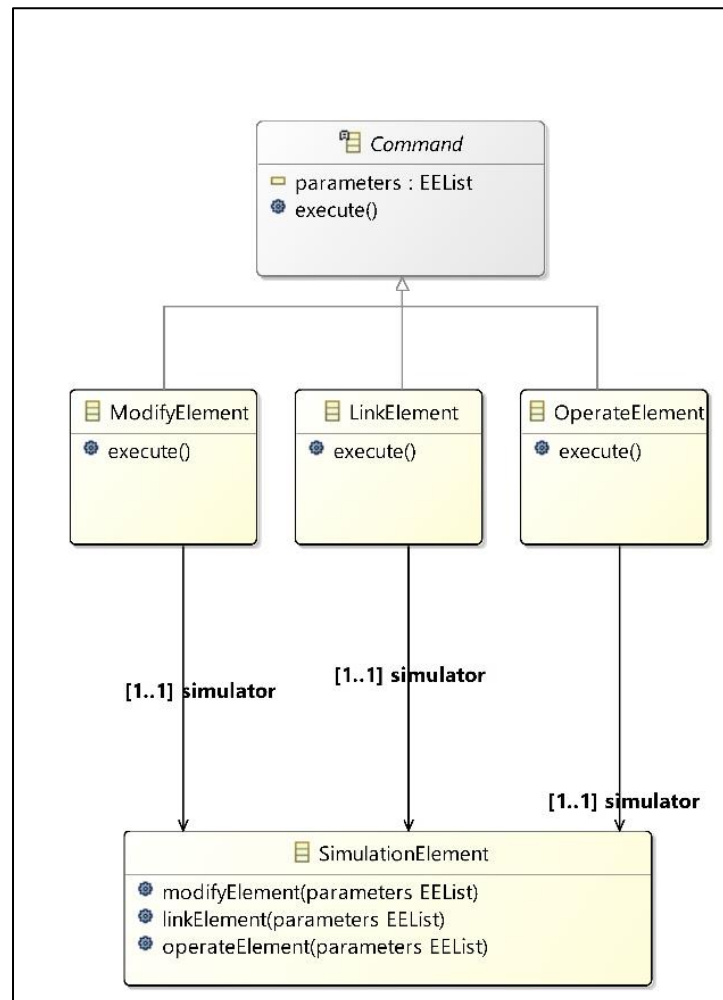
8.3.2 Comandos para elementos simulados

El segundo grupo de comandos son los comandos que se ejecutan directamente sobre elementos de la simulación cambiando el valor de sus atributos, ejecutando una o más de sus operaciones disponibles o efectuando cambios en las relaciones de estos elementos de simulación.

En la ilustración 7 se observa nuevamente la clase abstracta **Command**, y tres comandos que heredan de ésta, para este grupo de comandos el receptor es la clase

SimulationElement ya que que las operaciones que ejecutan estos comandos se encuentran en el elemento simulado o SimulationElement.

Ilustración 7 Comandos para elementos simulados



Fuente: El autor

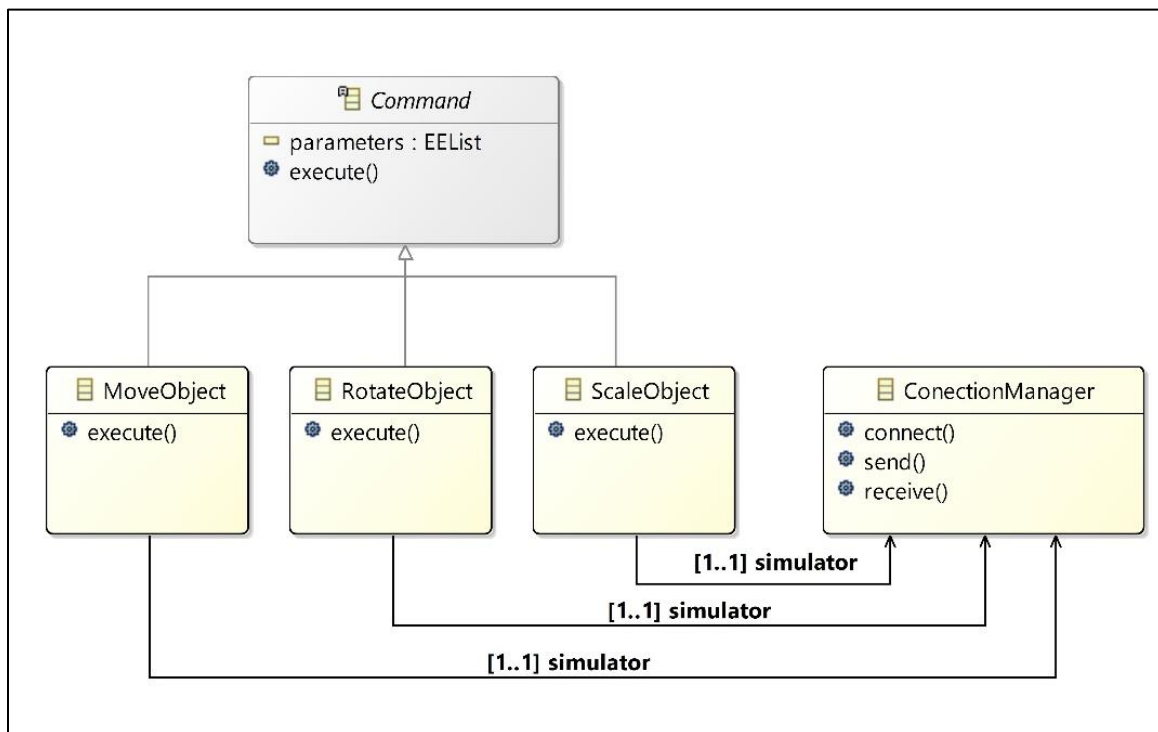
8.3.3 Comandos de visualización

Este grupo de comandos concretos está diseñado para gestionar las operaciones necesarias para la representación gráfica de la simulación. Estos comandos son ejecutados en el lado del cliente, ya que se relacionan con la visualización y manipulación de objetos en el entorno simulado. Ejemplos de estos comandos incluyen `MoveObject`, `RotateObject`, `ScaleObject` y `TransformObject`, los cuales efectúan cambios sobre objetos de la interfaz

gráfica de lado del cliente. Estos comandos permiten una interacción intuitiva y dinámica con el simulador, donde los usuarios pueden modificar el entorno visual sin afectar el estado de la simulación principal.

El tercer grupo de comandos se enfoca en operaciones específicas de la simulación, formando una base que se puede extender para adaptarse a diferentes tipos de simulaciones. Dentro de este grupo, se incluyen comandos como CreateObject, DeleteObject, ModifyObjectProperty y LinkObjects. Estos comandos afectan directamente los elementos del modelo de simulación y permiten configurar las relaciones y propiedades de los objetos simulados. Gracias a esta estructura extensible, el simulador puede adaptarse a distintos dominios de simulación sin necesidad de modificar el núcleo de la lógica de comandos.

Ilustración 8 Aplicación de patrón comando en el metamodelo, comandos de visualización



Fuente: El autor

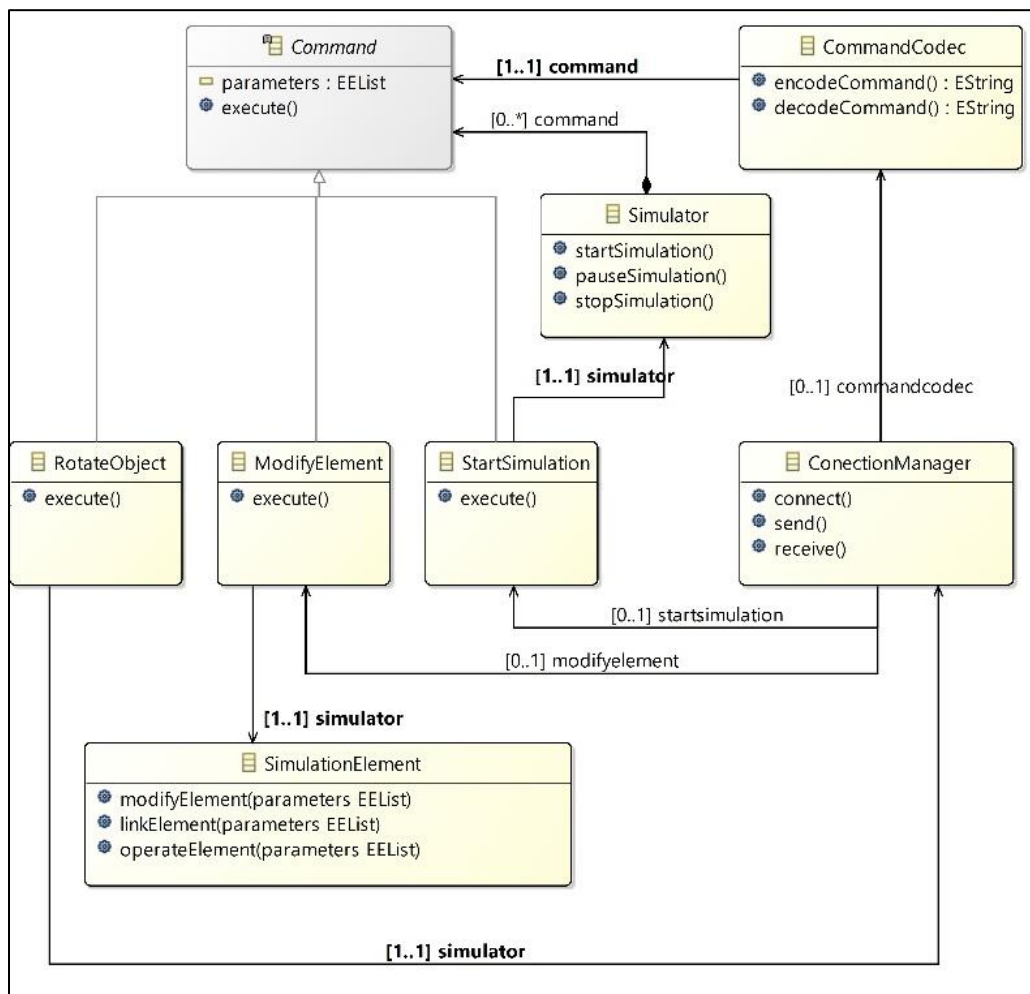
8.3.4 Envío y recepción de comandos

Para facilitar la comunicación y transmisión de comandos entre el cliente y el servidor, se desarrolló una clase `CommandCodec`, la cual proporciona las operaciones `encodeCommand` y `decodeCommand`. Esta clase permite codificar y decodificar los comandos en un formato representacional, como JSON o XML, para asegurar que puedan ser enviados y recibidos en un entorno distribuido de manera confiable y eficiente. En la ilustración, se detalla el funcionamiento de esta clase `CommandCodec`, la cual traduce los comandos desde y hacia objetos ejecutables, asegurando que la lógica de comandos sea independiente del canal de transmisión y que pueda integrarse con facilidad en un ambiente web interactivo.

8.3.5 Metamodelo de operaciones de simulación

Los tres grupos de comandos anteriormente descritos en conjunto permiten establecer el metamodelo completo capaz de atender las operaciones de simulación, en la Ilustración 9 se observa la aplicación completa del patrón comando en el metamodelo, allí se observa la clase comando de la que heredan los tres tipos de comandos,

Ilustración 9 Aplicación de patrón comando en el metamodelo, completo



Fuente: El autor

9 TRANSFORMACIÓN Y VALIDACIÓN

Este capítulo describe el proceso de transformación que se llevó a cabo, el cual está basado en un enfoque de Desarrollo Dirigido por Modelos (MDE), se detalla cómo se utilizaron tecnologías y herramientas como Eclipse Modeling Framework (EMF), Ecore y Acceleo para realizar las transformaciones específicas que generan código Java para el servidor y JavaScript para el cliente.

9.1 TRANSFORMACIÓN

La transformación de modelos fue un componente central en este proyecto, siguiendo los principios del Desarrollo Dirigido por Modelos (MDE) para generar artefactos de software a partir de modelos.

Para la implementación de las transformaciones en este proyecto, se seleccionó y utilizó un conjunto de tecnologías y herramientas del ecosistema MDE, centradas principalmente en el entorno de Eclipse. EMF proporcionó el soporte en tiempo de ejecución para estos modelos y las utilidades para la generación de código base. Específicamente, para las transformaciones del lado del servidor, se realizaron manipulaciones directas de los modelos Ecore dentro del entorno Eclipse.

Acceleo fue la herramienta de generación de código de modelo a texto (M2T) utilizada en el proyecto, se eligió Acceleo por su capacidad para transformar modelos EMF en diversos lenguajes textuales, su integración con OCL para la consulta de modelos, y sus herramientas de desarrollo dentro de Eclipse. Se empleó Acceleo para generar tanto el código Java del lado del servidor como el código JavaScript del lado del cliente a partir de los modelos Ecore definidos.

Como resultado de las transformaciones MDE realizadas, se generó código fuente en los siguientes lenguajes:

- **Java:** Para la lógica de negocio, persistencia y servicios del lado del servidor.
- **JavaScript:** Para la interfaz de usuario y la lógica del lado del cliente.

El proceso de transformación se dividió en dos flujos principales, uno para el lado del servidor y otro para el lado del cliente, ambos utilizando el modelo Ecore como fuente y Acceleo como motor de generación de código.

9.1.1 Transformaciones para el servidor

Para el lado del servidor, el proceso de transformación se centró en la generación de la estructura de las clases Java y otros componentes esenciales del proyecto, tomando como base el metamodelo Ecore. Inicialmente, se llevó a cabo la definición exhaustiva del metamodelo Ecore. En esta etapa, se especificaron detalladamente las EClass, EAttribute y EReference, elementos fundamentales que representan la arquitectura de datos y los diversos componentes que conforman el sistema del proyecto.

Posteriormente, se procedió con la creación de plantillas Acceleo con la extensión .mtl. Estas plantillas se desarrollaron de manera específica para interactuar de forma iterativa con los elementos definidos en el metamodelo Ecore del proyecto (simulation). El objetivo principal de estas plantillas fue generar de manera automatizada el código Java correspondiente a la estructura modelada, específicamente para los comandos. La estructura de la plantilla utilizada para generar una clase Java para un comando (Command) a partir del metamodelo se define de la siguiente manera:

```

[comment encoding = UTF-8 /]
[module generateJava('http://www.example.org/simulation')]

[template public generateCommand(aCommand : Command)]
package simulation.commands;

public class [aCommand.name/] {

    private String parameters = "[aCommand.parameters/]";

    public [aCommand.name/]() {
        System.out.println("Command created: " + "[aCommand.name/]" + " with parameters:
" + parameters);
    }

    public void execute() {
        System.out.println("Executing command: " + "[aCommand.name/]");
    }
}
[/template]

```

En esta plantilla específica del proyecto, se accede al nombre del Command mediante [aCommand.name/] y a sus parámetros con [aCommand.parameters/]. La directiva [file] (implícita en este contexto de generación de clases individuales) se usaría para especificar el archivo de salida con el nombre de la clase generada.

Las transformaciones Acceleo se ejecutaron desde Eclipse, utilizando el modelo .ecore del proyecto como entrada para producir los archivos .java para los comandos del simulador.

Adicionalmente, como se mencionó, se realizaron transformaciones directas en Ecore dentro de Eclipse. Esto implicó el uso de la API de EMF en Java para manipular mediante programación el modelo Ecore del proyecto, por ejemplo, para modificar o enriquecer el modelo antes de la generación final de código con Acceleo.

9.1.2 Transformación para el cliente

De forma similar, para el lado del cliente, se utilizó el mismo modelo Ecore del proyecto (simulation) como fuente para generar código JavaScript mediante plantillas Acceleo. El objetivo es producir artefactos como los observadores del cliente. La plantilla Acceleo específica del proyecto empleada para generar una clase JavaScript para un observador (ObserverRemoteSimulationElement) es:

```
[comment encoding = UTF-8 /]
[module generateJS('http://www.example.org/simulation')]

[template public generateObserverJS(aObserver : ObserverRemoteSimulationElement)]
class [aObserver.name/] {
    constructor() {
        console.log("Observer created for: " + "[aObserver.name/]");
    }

    update() {
        console.log("[aObserver.name/] received update.");
    }
}

const [aObserver.name/]Instance = new [aObserver.name/]();
[/template]
```

Esta plantilla itera sobre los elementos ObserverRemoteSimulationElement del metamodelo para definir clases JavaScript que actuarán como observadores en el cliente.

Se accede al nombre del observador mediante [aObserver.name/] para generar el nombre de la clase y una instancia de esta. Las transformaciones se ejecutaron para generar los archivos .js para la aplicación cliente.

9.1.3 Código específico

Se reconoce que el código generado automáticamente, aunque proporciona una base estructural sólida, necesita ser complementado manualmente con el fin de implementar la lógica de negocio específica y los comportamientos dinámicos detallados, especialmente para los elementos de simulación, cuyo comportamiento no se puede capturar completamente en los modelos estructurales Ecore dado que son específicos del contexto de simulación.

9.2 VALIDACIÓN

En las secciones anteriores se describió el proceso de transformación Model-to-Text (M2T) mediante plantillas Aceleo para generar artefactos de software tanto para el servidor como para el cliente, a partir del metamodelo de simulación remota interactiva. Esta sección presenta los resultados concretos de aplicar dicho metamodelo y sus principios de transformación para desarrollar un prototipo funcional de un simulador de enrutamiento en redes de comunicaciones, este caso de estudio sirve para validar la aplicabilidad del metamodelo propuesto.

9.2.1 Modelo específico para simulación de redes

El metamodelo general, con sus clases raíz como Simulator y sus patrones para el manejo de eventos (Observable, Observer, ObserverFactory) y operaciones (Command, CommandCodec), proveyó la estructura fundamental para el dominio específico de simulación de redes de comunicaciones.

A partir de esta base, la clase SimuladorRedes actúa como una especialización de la clase Simulator. Esta clase encapsula la lógica completa de la simulación de red, gestionando los dispositivos, sus interconexiones y el flujo de Unidades de Datos de Protocolo (PDU). Para representar los componentes de una red, se definieron clases concretas que heredan o

implementan la semántica de `SimulationElement`. Estos incluyen dispositivos de red como Encaminador, Computador, Conmutador y Concentrador, cada uno con atributos y comportamientos específicos, como la capacidad de enrutar paquetes por parte del Encaminador o ejecutar aplicaciones por parte del Computador. Otros elementos fundamentales son las Interfaces de Red, que representan los puertos de los dispositivos; las Conexiones de Red, que modelan los enlaces físicos o lógicos entre interfaces de dispositivos; y las Unidades de Datos de Protocolo (PDU), que simbolizan los paquetes de datos que transitan por la red simulada.

La gestión de eventos de simulación se implementó aplicando directamente el mecanismo de captura de eventos del metamodelo mediante el patrón Observador. En este esquema, elementos de simulación como las PDU, `ConexionRed`, Encaminador y Computador fueron diseñados como entidades `Observable`, capaces de notificar cambios en su estado. Complementariamente, se implementaron `Observer` concretos para diversos propósitos, tales como observadores para registrar eventos en consola o archivos de registro (por ejemplo, `ObservadorConexionTxt`) y observadores destinados a la interacción remota, responsables de enviar notificaciones de cambio al cliente para actualizar la visualización. Para la instanciación de estas familias de observadores, se utilizaron fábricas como `FabricaObservadoresTxt` y `FabricaObservadoresInteraccionRemota`, lo que demuestra la flexibilidad para extender la forma en que se manejan los eventos.

En cuanto al manejo de operaciones, la interacción entre el cliente y el servidor se gestionó mediante el patrón Comando. Se definieron `Command` específicos para acciones como crear y configurar dispositivos de red (por ejemplo, asignar una dirección IP a un Computador), conectar y desconectar dispositivos, e iniciar el envío de PDUs (como en la simulación de un "ping"). También se incluyeron comandos para la manipulación de la visualización en el cliente, derivados de eventos del usuario. Estos comandos se codificaron en formato JSON para su transmisión entre el cliente y el servidor a través de `WebSockets`, aplicando así el concepto de `CommandCodec`. El flujo de envío y recepción de comandos es un ejemplo de este mecanismo en acción.

Finalmente, utilizando plantillas Acceleo, se generaron artefactos de software. Para el servidor, se produjeron clases Java correspondientes a los comandos concretos (tales como StartSimulation, CreateObject, ModifyElement), tomando como entrada las definiciones del modelo Ecore específico de la simulación de redes. Para el cliente, se generaron archivos JavaScript para los observadores remotos y la lógica de manejo de la interfaz, derivados de las EClass correspondientes a los elementos visualizables y los eventos del servidor.

9.2.2 Pruebas y validación de prototipo

El prototipo de simulador de redes de comunicaciones fue sometido a diversas pruebas para validar su funcionalidad y el cumplimiento de los objetivos. Entre estas, las pruebas de conexión verificaron el establecimiento exitoso de la comunicación bidireccional mediante WebSockets entre el cliente y el servidor, así como el envío y recepción de mensajes de prueba.

Las pruebas del servidor se centraron en la lógica de simulación mediante la creación de escenarios de red, la configuración de dispositivos y el envío de Unidades de Datos de Protocolo (PDU). La correcta captura de eventos se validó a través de observadores que imprimían la traza de la simulación, lo que demostró el funcionamiento del SimuladorRedes y el sistema Observable/Observer.

Por el lado del cliente, las pruebas validaron la capacidad de la interfaz gráfica, desarrollada con HTML5, JavaScript y CreateJS, para capturar eventos del usuario, como la selección y ubicación de dispositivos. También se confirmó su habilidad para representar visualmente los elementos de la simulación y sus correspondientes animaciones.

Finalmente, se realizaron pruebas de integración, que implicaron el envío de comandos desde el cliente al servidor, desencadenados por acciones del usuario, y desde el servidor al cliente, originados por eventos en la simulación. Durante estas pruebas, se verificó la correcta codificación, transmisión, decodificación y ejecución de dichos comandos en ambos extremos del sistema.

9.2.3 Resultados

La aplicación del metamodelo en el desarrollo del simulador de redes de comunicaciones evidenció beneficios significativos. Primordialmente, se aceleró considerablemente el desarrollo del prototipo; la arquitectura predefinida por el metamodelo proveyó un marco robusto para la estructura cliente-servidor, los mecanismos de comunicación y la gestión de eventos. Esto permitió centrar los esfuerzos en las especificidades del dominio de simulación y agilizar la implementación mediante la generación de código.

El sistema resultante ofreció una alta interactividad, producto de la arquitectura cliente-servidor combinada con WebSockets y el patrón comando, asegurando que las acciones del usuario se reflejaran ágilmente y las actualizaciones de la simulación se visualizaran prontamente en una interfaz gráfica dinámica. Paralelamente, se logró un bajo uso de recursos computacionales en el cliente, ya que el procesamiento intensivo de la simulación se concentró en el servidor. El cliente gestionó la interfaz y la visualización, comunicándose eficientemente con el servidor mediante mensajes ligeros en formato JSON, lo que optimizó el rendimiento general y facilitó el acceso desde dispositivos con capacidades modestas. Esta clara separación funcional también demostró un procesamiento remoto efectivo.

Además, el diseño del simulador se caracterizó por una notable flexibilidad. El uso de patrones como Observador y Comando, junto con una arquitectura por componentes, facilitó la extensión del sistema con nuevas reacciones a eventos y operaciones sin necesidad de modificar el núcleo. Permitted también que la lógica de simulación, la de conexión y la de la vista pudieran evolucionar de forma independiente.

En resumen, la transformación del metamodelo abstracto en un simulador concreto no solo fue factible, sino que demostró la capacidad del metamodelo para guiar la construcción de sistemas de simulación remota interactiva que son eficientes y flexibles. Los resultados validaron la utilidad práctica del enfoque de Desarrollo Dirigido por Modelos (MDD) y los patrones arquitectónicos que este encapsula.

10 CONCLUSIONES

El metamodelo para simulación Web obtenido permite una estructura adaptable y escalable que responde a los actuales niveles de exigencia en múltiples tipos de escenarios de simulación, este diseño proporciona un marco que puede ser aplicado a diferentes escenarios de simulación, optimizando el uso de recursos y manteniendo altos niveles de interactividad y visualización progresiva.

En contextos con limitaciones de recursos y costos, las técnicas de procesamiento remoto y la arquitectura del metamodelo desarrolladas reducen el uso de ancho de banda y carga computacional en los dispositivos cliente, sin afectar las condiciones de alto nivel de interactividad, mejorando la accesibilidad a simulaciones complejas en entorno Web.

Se ha introducido una manera de integrar interactividad y procesamiento remoto. El uso de tecnologías y patrones de diseño, como el patrón comando, observador y fábrica abstracta, han mejorado la comunicación entre cliente y servidor reduciendo la latencia en tiempo de respuesta, lo que favorece la usabilidad y se refleja en una mejor experiencia del usuario en simulaciones interactivas con respecto a aplicaciones de simulación remota, propuestas en la literatura.

La implementación de visualización progresiva permite que las simulaciones interactivas en ambiente Web, se adapten de forma dinámica a las capacidades del dispositivo y a la disponibilidad de ancho de banda sin comprometer la experiencia del usuario.

11 APORTES Y TRABAJO FUTURO

Un paso importante es la adaptación y extensión del metamodelo desarrollado a otros dominios de simulación que requieren características avanzadas, como la realidad aumentada y la realidad virtual, esto permite explorar cómo el metamodelo se comporta en entornos de simulación inmersiva.

Incluir algoritmos de aprendizaje automático para optimizar la experiencia del usuario mediante la personalización y adaptación automática de las simulaciones en función del comportamiento del usuario y condiciones de baja latencia de la red.

Investigaciones futuras podrían enfocarse en optimizar las técnicas de procesamiento remoto y con ello la distribución de la carga entre servidor y cliente. Lo cual permite explorar cómo el metamodelo se comporta en entornos de arquitecturas basadas en la nube para maximizar la eficiencia y reducir la latencia.

Ampliar los estudios experimentales a escenarios de simulación con grandes volúmenes de usuarios concurrentes y alta carga de procesamiento. Esto implicaría evaluar el comportamiento de las transformaciones del metamodelo, bajo condiciones de estrés para verificar sus niveles de robustez.

Implementar herramientas adicionales que permitan a los desarrolladores y usuarios diseñar y personalizar entornos de simulación más intuitivos. Estas herramientas podrían incluir interfaces gráficas de usuario que faciliten la creación de modelos específicos a partir del metamodelo base.

12 REFERENCIAS

- Banks, J., Carson II, J. S., Nelson, B. L., & Nicol, D. M. (2014). *Discrete-Event System Simulation* (5ta ed.). Pearson.
- Booch, Grady., Rumbaugh, James., & Jacobson, Ivar. (2000). *The unified modeling language user guide* (6ta ed.). Addison-Wesley.
- Brambilla, M., Cabot, J., & Wimmer, M. (2017). *Model-Drive Software Engineering in Practice* (2da ed.). Morgan & Claypool publishers. <http://www.mdse-book.com>,
- Cassenti, D. N. (2018). Advances in Human Factors in Simulation and Modeling. *Advances in Intelligent Systems and Computing*, 780. <http://www.springer.com/series/11156>
- Cellier, F. E. ., & Kofman, Ernesto. (2006). *Continuous system simulation*. Springer Science+Business Media.
- Deniz, C. E. (2016). A Model Driven Approach to Web-based Traffic Simulation. *Proceedings of the Symposium on Theory of Modeling and Simulation: DEVS Integrative M&S Symposium (TMS-DEVS) - Mod4Sim(6th International Workshop on Model-driven Approaches for Simulation Engineering) track part of the Spring Simulation Multi-Conference*.
- Dominguez, X., Mantilla-Pérez, P., Gimenez, N., El-Sayed, I., Díaz Millán, M. A., & Arboleya, P. (2021). Web-based simulation environment for vehicular electrical networks. *Energies*, 14(19). <https://doi.org/10.3390/en14196087>
- Fujimoto, R. (2015). Parallel and distributed simulation. *Proceedings of the 2015 Winter Simulation Conference*.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1996). *Design Patterns*. Addison-Wesley.

- Hakiri, A., Berthou, P., & Gayraud, T. (2010). *Addressing the Challenge of Distributed Interactive Simulation With Data Distribution Service*.
- Heer, J., & Robertson, G. G. (2007). *Animated Transitions in Statistical Data Graphics*.
- Hernández Sampieri, R., Fernández Collado, C., María del Pilar Baptista Lucio, D., & Méndez Valencia Christian Paulina Mendoza Torres, S. (2014). *Metodología de la investigación* (6ta ed.). Mc Graw Hill Education.
- Jouault, F., Allilaire, F., Bézivin, J., & Kurtev, I. (2008). ATL: A model transformation tool. *Science of Computer Programming*, 72(1–2), 31–39.
<https://doi.org/10.1016/j.scico.2007.08.002>
- Knoblauch, A. (2024). *IVISIT: An Interactive Visual Simulation Tool for system simulation, visualization, optimization, and parameter management*.
- Koch, S., Reiche, F., & Heinrich, R. (2018). *Towards a Metamodel for Modular Simulation Environments*. <https://github.com/MoSimEngine>
- Law, A. M. . (2015). *Simulation modeling and analysis*. McGraw-Hill Education.
- Loreto, S., & Romano, S. Pietro. (2014). *Real-Time Communication with WebRTC*.
- Mell, P. M., & Grance, T. (2011). *The NIST definition of cloud computing*.
<https://doi.org/10.6028/NIST.SP.800-145>
- Menzel, S., Olhofer, M., & Sendhoff, B. (2010). *A METAMODEL-DRIVEN INTERACTIVE FRAMEWORK FOR A DESIGNER ASSISTANCE SYSTEM*.
- Metropolis, N., & Ulam, S. (1949). The Monte Carlo Method. *Journal of the American Statistical Association*, 44(247), 335–341.
- Minguelli, M. (2013). Interactividad e interacción. *Revista latinoamericana de tecnología educativa*, 1.

- Mondesire, S. C., Angelopoulou, A., Sirigampola, S., & Goldiez, B. (2018). *Combining virtualization and containerization to support interactive games and simulations on the cloud*. May. <https://doi.org/10.1016/j.simpat.2018.08.005>
- Mrozek, M., Sirenko, M., Foks-Ryznar, A., & Sawicki, B. (2022). Interactive Visualization of Agent-Based Pandemic Simulation in Web Browser. *2022 23rd International Conference on Computational Problems of Electrical Engineering, CPEE 2022*. <https://doi.org/10.1109/CPEE56060.2022.9919642>
- Ng, A., Grimm, H., Lezama, T., Persson, A., Andersson, M., & Jagstam, M. (2008). OPTIMISE: An Internet-Based Platform for Metamodel-Assisted Simulation Optimization. En *Advances in Communication Systems and Electrical Engineering* (pp. 281–296). Springer.
- Nielsen, J. (1993). *Usability Engineering*.
- OMG. (2003). *MDA Guide Version 1.0.1* (J. Miller & J. Mukerji, Eds.). Object Management Group.
- OMG. (2015). *OMG Unified Modeling Language (2.5)*. Object Management Group. <http://www.omg.org/spec/UML/2.5>
- OMG. (2019). *OMG Meta Object Facility (MOF) Core Specification (2.5.1)*. Object Management Group.
- Pham, H. (2022). A Web-based Interactive and Visualized Approach to Simulations of Operating Systems. *Proceedings - 2022 5th International Conference on Information and Computer Technologies, ICICT 2022*, 148–152. <https://doi.org/10.1109/ICICT55905.2022.00033>
- Pressman, R. S. (2010). *Ingenieria del Software. Un Enfoque Practico*. www.FreeLibros.me
- Qi Wu, David Bauer, Michael J. Doyle, & Kwan-Liu Ma. (2024). *Interactive Volume Visualization via Multi-Resolution Hash Encoding based Neural Representation*.

- Rorich, D., Bernhard, M., & Handte, T. (2014). *Webdemos: An Interactive, Web-Based Visualization and Simulation Framework for Open Access*. [IEEE].
- Rorich, D., Bernhard, M., Handte, T., & Brink, S. (2014). Webdemos: An Interactive, Web-Based Visualization and Simulation Framework for Open Access. *Institute of Telecommunications University of Stuttgart*.
- Rumbaugh, James., Jacobson, Ivar., & Booch, Grady. (2000). *The unified modeling language reference manual*. Addison-Wesley Longman.
- Soler-Adillon, J. (2012). *Principios de diseño de interacción para sistemas interactivos*.
- Stachowiak, Herbert. (1973). *Allgemeine Modelltheorie*. Springer-Verlag.
- St-Aubin, B. (2022). *A web based architecture to operationalize geospatial simulation environments*.
- Steinberg, D., Budinsky, F., Paternostro, M., & Merks, E. (2009). *Eclipse Modeling Framework*. Addison-Wesley.
- Steuer, J. (1992). Defining Virtual Reality: Dimensions Determining Telepresence. *Journal of Communication*, 42(4), 73–93. <https://doi.org/10.1111/j.1460-2466.1992.tb00812.x>
- Tanenbaum, A. (2007). *Distributed Systems* (2da ed.). Pearson. www.minix3.org.This
- Taylor, S., Khan, A., Morse, K., Tolk, A., Yilmaz, L., Zander, J., & Mosterman, P. (2015). Grand challenges for modeling and simulation: Simulation everywhere - From cyberinfrastructure to clouds to citizens. *SIMULATION*, 91, 648–665. <https://doi.org/10.1177/0037549715590594>
- Universidad Europea. (2024). *¿Qué es el diseño responsive? | Blog CC*. <https://creativecampus.universidadeuropea.com/blog/que-es-diseno-responsive/>
- Velichko, A. (2018). *Interactive Simulation Software for Multi-Regional Model of Freight Transportation*. <http://ceur-ws.org>

- Vidal, G. A. (2016). *Análisis de herramientas de generación automática de código Android a partir de modelos*. Universidad Nacional de la Plata.
- Warmer, J. B., Kleppe, A. G., & Bast, Wim. (2003). *MDA Explained: The Model Driven ArchitectureTM: Practice and Promise* Bast, Wim. Addison-Wesley Professional.
- Zeigler, B., Prähofer, H., & Kim, T. G. (2000). *Theory of Modeling and Simulation: Integrating Discrete Event and Continuous Complex Dynamic Systems*. 2.
- Zhou, W., Tang, K., & Jia, J. (2018). S-LPM: segmentation augmented light-weighting and progressive meshing for the interactive visualization of large man-made Web3D models. *World Wide Web*, 21(5), 1425–1448. <https://doi.org/10.1007/s11280-018-0610-1>